Der Subnetting Algorithmus

Stephan Epp
M.Sc.

18. Oktober 2026

Zusammenfassung

Der Subnetting Algorithmus `sbntgg` (Kommandozeilenprogramm `subnetplan`) teilt einen zugewiesenen IPv4- oder IPv6-Adressbereich automatisch in überlappungsfreie Subnetze für Mitarbeiter- und Ressourcengruppen auf und erzeugt aus einer Zugriffsmatrix Firewall-Regeln für `iptables`, `nftables` und Cisco-ACLs. Die Arbeit definiert das Problem formal, beweist die Korrektheit der Zuteilung (Ausrichtung, Überlappungsfreiheit, Kapazitätsgarantie), zeigt, dass die Strategie „größte Blöcke zuerst“ lückenlos und unter allen Anordnungen optimal ist, charakterisiert die freien Restbereiche exakt und beweist, dass sich jede Wachstumsreserve ohne Umnummerierung nutzen lässt, sobald ihre Gesamtgröße in den freien Adressraum passt. Für die Firewall-Regeln wird ein Default-Deny-Satz bewiesen. Die Laufzeit des Kerns beträgt $\Theta(n \log n + a)$ im Vergleichsmodell und ist damit optimal. Die zunächst nur vorgeschlagenen Erweiterungen sind in Version 1.1.0 umgesetzt: Planung mit Bestandsnetzen und mehreren Pools (exakte Packbarkeitsbedingung), Mindestgrößen und eigener Overhead, zweistufige Planung mit Standortblöcken, inkrementelle Erweiterung und Verdopplung an Ort und Stelle, Regelgruppierung, Rückverkehr bei Cisco-ACLs sowie eine Rückübersetzung der erzeugten Regeln als Korrektheitstest. Der Richtlinienvergleich über den Subgraph Algorithmus wird durch eine exakte Prüfung abgesichert, weil dessen Signaturvergleich, wie die Arbeit zeigt, nur eine Heuristik ist. Ein ausführlich erklärtes Konfigurationsbeispiel, eine Einführung in das Subnetting für Schülerinnen und Schüler sowie ein umfangreicher Ausblick ergänzen die Arbeit.

Inhaltsverzeichnis

1 Einführung **1**

1.1 Problemstellung . . . 1
1.2 Motivation . . . 1
1.3 Aufbau der Arbeit . . . 2

2 Grundlagen **3**

2.1 Subnetting leicht erklärt . . . 3
2.2 Was ist eine IP-Adresse? . . . 3
2.3 Das Dualsystem in einer Minute . . . 3
2.4 Netz und Host: Straße und Hausnummer . . . 3
2.5 Drei besondere Adressen . . . 4
2.6 Das Rezept: Wie groß muss das Netz sein? . . . 4
2.7 Die Schokoladentafel: Warum Zweierpotenzen? . . . 5
2.8 Wozu das Ganze? . . . 5
2.9 Formale Definitionen . . . 6
2.10 Grundidee . . . 7
2.11 Blockgrößen als Zweierpotenzen . . . 7
2.12 Größte Blöcke zuerst (VLSM) . . . 7
2.13 Firewall-Regeln aus der Zugriffsmatrix . . . 8

3 Algorithmus **9**

3.1 Algorithmus-Beschreibung . . . 9
3.2 Bedarf und Präfixlänge . . . 9
3.3 VLAN-Vergabe . . . 11
3.4 Reihenfolge und Zuteilung . . . 11
3.5 Freie Bereiche . . . 13
3.6 Firewall-Regeln . . . 15
3.7 Implementierung . . . 16
3.8 Bedarf und Präfixlänge . . . 17
3.9 VLAN-Vergabe . . . 18
3.10 Zuteilung der Subnetze . . . 18
3.11 Freie Bereiche . . . 19
3.12 Firewall-Regeln . . . 20
3.13 Validierung der Konfiguration . . . 20
3.14 Module der Erweiterungen . . . 21

4 Analyse **23**
4.1 Laufzeit . . . 23
4.2 Optimalität des Kerns . . . 24
4.3 Speicher . . . 24
4.4 Alternative: Buddy-System . . . 25
4.5 Gesamtkorrektheit . . . 25
4.6 Experimentelle Überprüfung . . . 25

5 Verwendungsbeispiele **29**
5.1 Das Konfigurationsbeispiel im Detail . . . 29
5.2 Die Datei . . . 29
5.3 YAML in zwei Minuten . . . 30
5.4 Die vier Kopfzeilen . . . 30
5.5 Die fünf Gruppen . . . 30
5.6 Schritt 1: Bedarf und Präfixlänge . . . 31
5.7 Schritt 2: Reihenfolge . . . 31
5.8 Schritt 3: Der Zeiger . . . 31
5.9 Schritt 4: VLAN-IDs . . . 32
5.10 Das Ergebnis . . . 32
5.11 Nutzbar, Bedarf und Reserve . . . 33
5.12 Ein Blick in die Binärdarstellung . . . 33
5.13 Freie Bereiche und Wachstum . . . 33
5.14 Die Firewall-Regeln . . . 33
5.15 Varianten ohne Bearbeiten der Datei . . . 35
5.16 Typische Fehler . . . 35
5.17 Eigene Konfiguration ableiten . . . 36
5.18 Weitere Code-Beispiele . . . 36
5.19 Maschinenlesbare Ausgabe . . . 36
5.20 Verwendung als Python-Bibliothek . . . 36
5.21 IPv6 . . . 37
5.22 Wiederholte Zugriffsziele . . . 37

6 Experimentelle Ergebnisse **38**
6.1 Versuchsaufbau und Reproduzierbarkeit . . . 38
6.2 Adressbedarf, Auslastung und Packbarkeit . . . 39
6.3 Erweiterung und hierarchische Planung . . . 40
6.4 Firewall-Regeln und Richtlinien . . . 41
6.5 Nicht abgeschlossene Experimente und Aussagegrenzen . . . 41

7 Zusammenfassung **43**
7.1 Anwendungen . . . 44
7.2 Erweiterungen: Umsetzung und Ausblick . . . 44
7.3 Verbesserungen am bestehenden Programm . . . 45
7.4 Weniger Regeln durch Gruppierung . . . 46
7.5 Erweiterungen der Planung . . . 46
7.6 Erweiterungen der Firewall-Erzeugung . . . 50
7.7 Verbindung zum Subgraph Algorithmus . . . 50
7.8 Integration und Ausgabeformate . . . 52
7.9 Qualitätssicherung und formale Verifikation . . . 52

7.10 Offene theoretische Fragen 52
7.11 Priorisierte Übersicht 53
7.12 Implementierungshinweis 53

Kapitel 1

Einführung

Der Subnetting Algorithmus `sbntgg` ist ein deterministischer Algorithmus zur automatischen Planung von IP-Netzen. Aus einer Konfigurationsdatei mit einem zugewiesenen Adressbereich (dem *Basisnetz*) und einer Liste von Gruppen (zum Beispiel „Entwicklung“, „HR“, „Drucker“) berechnet er für jede Gruppe ein eigenes Subnetz mit VLAN-ID, Gateway und Wachstumsreserve. Optional erzeugt er aus der Zugriffsmatrix der Gruppen ein Regelwerk für Firewalls.

1.1 Problemstellung

Gegeben sind ein Basisnetz B (IPv4 oder IPv6), n Gruppen $g_1, \ldots, g_n$ mit jeweils h_i Hosts, ein Wachstumsfaktor $\gamma \geq 1$, eine Strategie (`vlsm` oder `fixed`), eine Gateway-Position (`first`, `last` oder `none`) sowie pro Gruppe optional eine VLAN-ID, eine Priorität und eine Menge von Gruppen, auf die sie zugreifen darf.

Ziel: Bestimme für jede Gruppe ein Subnetz innerhalb von B, sodass alle Subnetze paarweise überlappungsfrei sind, jedes Subnetz mindestens $\lceil \gamma h_i \rceil$ nutzbare Host-Adressen besitzt und jede Gruppe eine eindeutige VLAN-ID erhält.

Ergebnis:

- Wenn die Summe der benötigten Blockgrößen in B passt: Gib den Plan aus (Subnetze, Gateways, Host-Bereiche, freie Restbereiche) und, falls gewünscht, Firewall-Regeln.
- Wenn die Summe nicht in B passt: Brich mit einer Fehlermeldung zum Platzmangel ab und gib keinen Plan aus (Satz 3 zeigt, dass dann auch jedes andere Verfahren scheitern muss).
- Wenn die Konfiguration ungültig ist (unbekannte Schlüssel, doppelte Namen oder VLAN-IDs, unbekannte Zugriffsziele): Brich ebenfalls mit einer Fehlermeldung ab.
- Wenn `access` leer ist: Die Gruppe erhält keinen Zugriff auf interne Netze.

1.2 Motivation

Die manuelle Planung von Subnetzen ist fehleranfällig: Netze überlappen sich, Blockgrenzen werden falsch berechnet, VLAN-IDs doppelt vergeben, und das Wachstum der Gruppen wird vergessen. Gleichzeitig wird die Zugriffsmatrix „wer darf auf wen“ meist getrennt vom Adressplan in Firewall-Regeln übersetzt, wodurch Plan und Regeln auseinanderlaufen.

`sbntgg` berechnet beides aus einer einzigen Quelle. Die Grundlagen sind das Subnetting nach [1], die klassenlose Adressierung (CIDR) nach [4], variable Subnetzmasken nach [2] und die IPv6-Adressarchitektur nach [5].

Eine besondere Verbindung besteht zum Subgraph Algorithmus: Die Zugriffsmatrix ist die Adjazenzmatrix eines gerichteten Graphen über den Gruppen (Abschnitt 7.7). Der Vergleich zweier Richtlinien ist damit eine Aufgabe für den Subgraph Algorithmus [14].

1.3 Aufbau der Arbeit

Abschnitt 2 erklärt das Subnetting zuerst anschaulich und dann formal. Abschnitt 3 beschreibt den Algorithmus samt Implementierung und beweist seine Korrektheit. Abschnitt 4 analysiert Laufzeit, Optimalität und Speicher. Abschnitt 5 erläutert das Konfigurationsbeispiel und weitere Code-Beispiele. Abschnitt 6 wertet die reproduzierbaren Experimente aus. Abschnitt 7 fasst die Arbeit zusammen, beschreibt die umgesetzten Erweiterungen samt Beweisen und Prüfergebnissen und nennt die noch offenen Punkte.

Kapitel 2

Grundlagen

2.1 Subnetting leicht erklärt

Dieser Abschnitt ist so geschrieben, dass er auch in einer Realschulklasse (etwa im Informatikunterricht ab Klasse 8) verstanden werden kann. Es wird nur Rechnen mit Zweierpotenzen benötigt.

2.2 Was ist eine IP-Adresse?

Jedes Gerät in einem Netzwerk (Computer, Handy, Drucker) braucht eine Adresse, damit Nachrichten ankommen, so wie ein Haus eine Hausnummer braucht, damit der Briefträger es findet. Eine IPv4-Adresse besteht aus vier Zahlen zwischen 0 und 255, getrennt durch Punkte, zum Beispiel `10.20.1.130`.

Warum nur bis 255? Jede der vier Zahlen besteht aus 8 *Bit*. Ein Bit ist eine Stelle, die 0 oder 1 sein kann. Mit 8 Bit gibt es $2^8 = 256$ Möglichkeiten, also die Zahlen 0 bis 255. Eine ganze Adresse hat $4 \cdot 8 = 32$ Bit und damit $2^{32} = 4\,294\,967\,296$ mögliche Adressen.

2.3 Das Dualsystem in einer Minute

Computer rechnen mit Zweierpotenzen. Jede Stelle einer Dualzahl hat einen Wert, der doppelt so groß ist wie die Stelle rechts davon:

Stellenwert	128	64	32	16	8	4	2	1
10	0	0	0	0	1	0	1	0
130	1	0	0	0	0	0	1	0
192	1	1	0	0	0	0	0	0
224	1	1	1	0	0	0	0	0

Zum Beispiel ist $130 = 128 + 2$, also steht eine 1 bei 128 und eine 1 bei 2.

2.4 Netz und Host: Straße und Hausnummer

Eine Adresse besteht aus zwei Teilen: einem *Netzteil* (die Straße) und einem *Hostteil* (die Hausnummer in dieser Straße). Der Zusatz hinter dem Schrägstrich, etwa `/26`, sagt, wie

viele der 32 Bit zum Netzteil gehören. Das ist die *Präfixlänge*. Die übrigen Bit sind der Hostteil.

Beim Netz `10.20.1.128/26` gehören 26 Bit zur Straße. Für die Hausnummern bleiben $32 - 26 = 6$ Bit, also $2^6 = 64$ Adressen. Wichtig ist: **Je größer die Zahl hinter dem Schrägstrich, desto kleiner das Netz.**

Präfix	Hostbits	Adressen im Netz	Subnetzmaske
`/24`	8	256	`255.255.255.0`
`/25`	7	128	`255.255.255.128`
`/26`	6	64	`255.255.255.192`
`/27`	5	32	`255.255.255.224`
`/28`	4	16	`255.255.255.240`
`/29`	3	8	`255.255.255.248`
`/30`	2	4	`255.255.255.252`

Die *Subnetzmaske* ist dieselbe Information anders geschrieben: so viele Einsen von links, wie das Präfix lang ist. Bei `/26` sind das 26 Einsen, im letzten Byte `11000000`, das ist $128 + 64 = 192$.

2.5 Drei besondere Adressen

In jedem Netz sind einige Adressen nicht für normale Geräte frei:

- Die **Netzadresse** (die erste Adresse) ist das Straßenschild. Sie benennt das ganze Netz.
- Die **Broadcast-Adresse** (die letzte Adresse) ist die Lautsprecherdurchsage „an alle in dieser Straße“.
- Das **Gateway** ist das Tor aus der Straße hinaus in andere Straßen und ins Internet. Man legt es üblicherweise auf die erste oder letzte freie Adresse.

Von den Adressen eines Netzes gehen also bis zu drei für Geräte verloren. Ein `/28` hat 16 Adressen, davon sind $16 - 3 = 13$ für Geräte nutzbar.

2.6 Das Rezept: Wie groß muss das Netz sein?

Beispiel: In der Gruppe „Drucker“ stehen 12 Geräte, und es soll 25 % Platz für Wachstum geben (Faktor 1,25).

1. Geräte: 12.
2. Mit Wachstum: $12 \cdot 1{,}25 = 15$ (bei Kommazahlen wird aufgerundet).
3. Plus drei Sonderadressen: $15 + 3 = 18$.
4. Die nächste Zweierpotenz, die mindestens 18 ist: $2^5 = 32$ (denn $2^4 = 16$ ist zu klein).
5. Hostbits sind 5, also ist das Präfix $32 - 5 = 27$. Das Netz ist ein `/27`.

Probe: Ein `/27` hat 32 Adressen, davon sind $32 - 3 = 29$ nutzbar. Das ist mehr als 15. Ein `/28` hätte nur 13 nutzbare Adressen gehabt, zu wenig.

2.7 Die Schokoladentafel: Warum Zweierpotenzen?

Stell dir eine Schokoladentafel vor, die nur in der Mitte gebrochen werden kann. Jede Hälfte hat wieder eine Mitte, und so fort. So entstehen immer Stücke mit $1, 2, 4, 8, \ldots$ Teilen, niemals Stücke mit 3 oder 5 Teilen. Genau so werden Adressbereiche geteilt (Abbildung 2.1): Jede Halbierung erhöht das Präfix um 1.

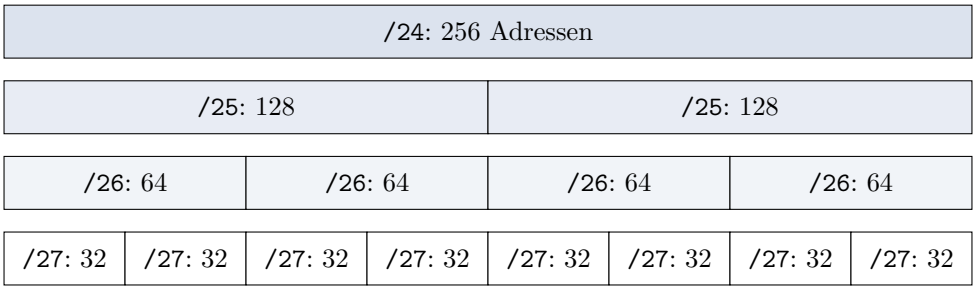


Abbildung 2.1: Jede Halbierung verdoppelt die Anzahl der Blöcke und halbiert ihre Größe. Ein Block beginnt immer an einer „Rille“.

Daraus folgen zwei Regeln, die der Algorithmus befolgt:

1. Ein Netz darf nur dort beginnen, wo eine „Rille“ ist (*Ausrichtung*). Ein `/27` mit 32 Adressen kann bei den Adressen $0, 32, 64, \ldots$ beginnen, aber nicht bei 40.

2. Man legt **die großen Stücke zuerst** und die kleinen danach. Dann bleiben keine Lücken, weil jedes kleinere Stück genau dort beginnen kann, wo das vorige aufhört.

Beispiel 1 (Eine Schule mit `192.168.0.0/24`)**.** Die Schule hat 256 Adressen. Es gibt 100 Schüler-PCs, 25 Lehrer-PCs und 10 Verwaltungs-PCs, ohne Wachstumsreserve und mit Gateway. Das Programm rechnet:

Gruppe	Geräte	+3	Zweierpotenz	Präfix	Netz
Schüler	100	103	$128 = 2^7$	`/25`	`192.168.0.0/25`
Lehrer	25	28	$32 = 2^5$	`/27`	`192.168.0.128/27`
Verwaltung	10	13	$16 = 2^4$	`/28`	`192.168.0.160/28`

Die Schüler (das größte Netz) kommen zuerst: Adressen 0 bis 127. Danach folgen die Lehrer ab 128 bis 159 und die Verwaltung ab 160 bis 175. Belegt sind $128 + 32 + 16 = 176$ von 256 Adressen (68,75 %), frei sind die 80 Adressen `192.168.0.176/28` und `192.168.0.192/26`.

2.8 Wozu das Ganze?

– **Ordnung:** Jede Abteilung hat ihren eigenen Bereich, und man erkennt an der Adresse, wohin ein Gerät gehört.

– **Sicherheit:** Gäste-WLAN und Server liegen in verschiedenen Netzen. Eine Firewall entscheidet, wer von wo nach wo darf.

– **Geschwindigkeit:** Eine Durchsage (Broadcast) erreicht nur die Geräte im eigenen kleinen Netz und nicht den ganzen Betrieb.

– **Wachstum:** Freie Bereiche bleiben für neue Gruppen übrig.

Kontrollfragen. (1) Wie viele Adressen hat ein `/29`? ($2^3 = 8$.) (2) Wie viele davon sind für Geräte nutzbar, wenn es Netzadresse, Broadcast und Gateway gibt? ($8 - 3 = 5$.) (3) Welches Präfix braucht man für 50 Geräte ohne Reserve? ($50 + 3 = 53 \le 64 = 2^6$, also `/26`.)

2.9 Formale Definitionen

Im Folgenden bezeichnet $w \in \{32, 128\}$ die Adressbreite (IPv4 beziehungsweise IPv6). Adressen werden als ganze Zahlen identifiziert.

Definition 1 (Adressraum und Block)**.** Der Adressraum ist $\mathcal{A}_w = \{0, 1, \ldots, 2^w - 1\}$. Für eine Präfixlänge $p \in \{0, \ldots, w\}$ und eine Adresse $a \in \mathcal{A}_w$ mit $2^{w-p} \mid a$ heißt

$$N(a, p) = \{a, a + 1, \ldots, a + 2^{w-p} - 1\}$$

ein *Block* (CIDR-Block) der Präfixlänge p. Er hat die Größe $|N(a,p)| = 2^{w-p}$, die *Netzadresse* a und die *letzte Adresse* $a + 2^{w-p} - 1$. Die Bedingung $2^{w-p} \mid a$ heißt *Ausrichtung*.

Lemma 1 (Laminarität)**.** Zwei Blöcke $N_1 = N(a_1, p_1)$ und $N_2 = N(a_2, p_2)$ mit $p_1 \ge p_2$ sind entweder disjunkt oder es gilt $N_1 \subseteq N_2$. Bei $p_1 = p_2$ sind sie gleich oder disjunkt.

Beweis. Sei $q_i = w - p_i$, also $q_1 \le q_2$. Wegen der Ausrichtung ist $N_i = \{x : \lfloor x/2^{q_i} \rfloor = a_i/2^{q_i}\}$. Gilt $N_1 \cap N_2 \neq \emptyset$, so gibt es ein x in beiden Mengen. Für jedes $y \in N_1$ gilt $\lfloor y/2^{q_1} \rfloor = a_1/2^{q_1} = \lfloor x/2^{q_1} \rfloor$. Mit der Identität $\lfloor \lfloor z/m \rfloor / k \rfloor = \lfloor z/(mk) \rfloor$ für natürliche m, k folgt

$$\left\lfloor \frac{y}{2^{q_2}} \right\rfloor = \left\lfloor \frac{\lfloor y/2^{q_1} \rfloor}{2^{q_2 - q_1}} \right\rfloor = \left\lfloor \frac{\lfloor x/2^{q_1} \rfloor}{2^{q_2 - q_1}} \right\rfloor = \left\lfloor \frac{x}{2^{q_2}} \right\rfloor = \frac{a_2}{2^{q_2}},$$

also $y \in N_2$. Somit $N_1 \subseteq N_2$. Bei $p_1 = p_2$ gilt zusätzlich $N_2 \subseteq N_1$ mit demselben Argument, also $N_1 = N_2$. □

Definition 2 (Gruppe, Konfiguration)**.** Eine *Gruppe* ist ein Tupel $g = (\text{name}, h, v, \pi, \text{acc})$ mit Namen, Hostzahl $h \in \mathbb{N}_{\ge 1}$, optionaler VLAN-ID $v \in \{1, \ldots, 4094\}$, Priorität $\pi \in \mathbb{N}_0$ (Standard 0) und einer Liste acc von Gruppennamen (Standard: leer). Eine *Konfiguration* besteht aus einem Basisnetz $B = N(a_B, p_B)$, einer Liste $g_1, \ldots, g_n$ ($n \ge 1$) von Gruppen mit paarweise verschiedenen Namen, einem Wachstumsfaktor $\gamma \ge 1$, einer Strategie $\in \{\text{vlsm}, \text{fixed}\}$ und einer Gateway-Position $\in \{\text{first}, \text{last}, \text{none}\}$. Die Reihenfolge der Liste ist die *Dateireihenfolge*.

Definition 3 (Bedarf, Blockexponent)**.** Der *Overhead* ist $o = 2 + [\text{gw} \neq \text{none}]$ für IPv4 und $o = 1 + [\text{gw} \neq \text{none}]$ für IPv6, wobei $[\cdot]$ die Iverson-Klammer ist (IPv6 kennt keine Broadcast-Adresse). Der *Bedarf* der Gruppe g_i ist

$$r_i = \lceil \gamma h_i \rceil + o.$$

Der *Blockexponent* ist $k_i = \lceil \log_2 r_i \rceil$, die *Blockgröße* $s_i = 2^{k_i}$ und die *Präfixlänge* $p_i = w - k_i$. Ferner sei $S = \sum_{i=1}^{n} s_i$.

Definition 4 (Subnetz)**.** Ein *Subnetz* der Gruppe g_i ist ein Block $N = N(a, p)$ mit $s = 2^{w-p}$ zusammen mit folgenden ausgezeichneten Adressen (Netzadresse a und, nur IPv4, Broadcast $a + s - 1$):

Gateway-Position	Gateway	erster Host	letzter Host (IPv4)	letzter Host (IPv6)
`none`	–	$a+1$	$a+s-2$	$a+s-1$
`first`	$a+1$	$a+2$	$a+s-2$	$a+s-1$
`last`	$a+s-2$ (IPv4), $a+s-1$ (IPv6)	$a+1$	$a+s-3$	$a+s-2$

Die Anzahl *nutzbarer Hosts* ist $u = (\text{letzter Host}) - (\text{erster Host}) + 1$.

Lemma 2 (Nutzbare Hosts)**.** Für jedes Subnetz der Größe s gilt $u = s - o$. Ist $s \geq r_i$, so gilt $u \geq \lceil \gamma h_i \rceil \geq 1$.

Beweis. IPv4 ohne Gateway: $u = (a+s-2)-(a+1)+1 = s-2 = s-o$. IPv4 mit Gateway (`first`): $u = (a+s-2)-(a+2)+1 = s-3 = s-o$; (`last`): $u = (a+s-3)-(a+1)+1 = s-3 = s-o$. IPv6 ohne Gateway: $u = (a+s-1)-(a+1)+1 = s-1 = s-o$; mit Gateway: $u = s-2 = s-o$ in beiden Positionen. Ist $s \geq r_i = \lceil \gamma h_i \rceil + o$, so folgt $u = s - o \geq \lceil \gamma h_i \rceil$, und $\lceil \gamma h_i \rceil \geq 1$ wegen $\gamma h_i \geq h_i \geq 1$. □

Definition 5 (Plan)**.** Ein *Plan* ist eine Liste von Subnetzen $(N_1, \ldots, N_n)$ (in Adressreihenfolge ausgegeben) mit zugehörigen VLAN-IDs, zusammen mit der Liste der *freien Bereiche*, einer Zerlegung von $B \setminus \bigcup_i N_i$ in Blöcke. Die *belegten Adressen* sind $\sum_i |N_i|$, die *freien Adressen* sind $F = |B| - \sum_i |N_i|$ und die *Auslastung* ist $100 \cdot \sum_i |N_i| / |B|$ Prozent.

Definition 6 (Zugriffsmatrix und Zugriffsgraph)**.** Die *Zugriffsmatrix* ist die Matrix $Z \in \{0,1\}^{n \times n}$ mit $Z_{ij} = 1$ genau dann, wenn der Name von g_j in $\mathrm{acc}(g_i)$ vorkommt. Der *Zugriffsgraph* ist der gerichtete Graph $G_Z = (V, E)$ mit $V = \{g_1, \ldots, g_n\}$ und $E = \{(g_i, g_j) : Z_{ij} = 1\}$ (vgl. die Definition der Adjazenzmatrix im Subgraph Algorithmus). Die Menge der *Zugriffspaare* ist $P = \{(i,j) : Z_{ij} = 1,\ i \neq j\}$.

2.10 Grundidee

Der Algorithmus basiert auf drei zentralen Konzepten.

2.11 Blockgrößen als Zweierpotenzen

Jede Gruppe erhält den kleinsten Block, der ihren Bedarf fasst. Blöcke sind nach Lemma 1 entweder disjunkt oder ineinander enthalten. Das macht die Überlappungsfreiheit zu einer reinen Frage der Startadressen.

2.12 Größte Blöcke zuerst (VLSM)

Variable Length Subnet Masks (VLSM) bedeutet, dass Subnetze unterschiedlich groß sein dürfen. Werden die Blöcke nach absteigender Größe hintereinander gelegt, ist jede Startadresse automatisch ausgerichtet (Satz 2). Es entstehen keine Lücken und der Platzbedarf ist minimal (Satz 5).

Beispiel 2. Für $n = 3$ Gruppen mit Blockgrößen $32, 256, 64$ (aufsteigend in Dateireihenfolge) liefert die Sortierung die Reihenfolge $256, 64, 32$ und damit die Startadressen $0, 256, 320$. In aufsteigender Reihenfolge $(32, 64, 256)$ lägen die Startadressen bei $0, 64, 256$ und der letzte Block endet erst bei 512 statt bei 352.

2.13 Firewall-Regeln aus der Zugriffsmatrix

Alles, was nicht ausdrücklich in `access` freigegeben ist, wird verworfen (Default-Deny). Jedes Zugriffspaar $(i, j) \in P$ ergibt genau eine Freigabe vom Subnetz N_i in das Subnetz N_j (Satz 10).

Kapitel 3

Algorithmus

3.1 Algorithmus-Beschreibung

Der Algorithmus arbeitet in sechs Schritten:

Schritt 1: Einlesen und Validieren

Lies die YAML- oder JSON-Datei, wende die Kommandozeilen-Überschreibungen (`-base-network`, `-growth-factor`, `-strategy`, `-gateway`) an und prüfe: nur bekannte Schlüssel, $\gamma \geq 1$, Basisnetz streng gültig (Hostbits null), $h_i \geq 1$, VLAN-IDs in $\{1, \ldots, 4094\}$ und paarweise verschieden, Gruppennamen eindeutig und nur aus Buchstaben, Ziffern, `_`, `-` und Leerzeichen, alle `access`-Einträge nennen existierende Gruppen.

Schritt 2: Bedarf und Präfixlänge

Berechne für jede Gruppe $r_i = \lceil \gamma h_i \rceil + o$ und die kleinste Präfixlänge $p_i = w - \lceil \log_2 r_i \rceil$.

Schritt 3: VLAN-IDs und Reihenfolge

Übernimm explizite VLAN-IDs, vergib fehlende automatisch $(10, 20, 30, \ldots)$, und bestimme die Zuteilungsreihenfolge: bei `vlsm` nach (p_i, π_i) lexikographisch aufsteigend, bei `fixed` nach π_i mit einheitlicher Präfixlänge $\min_i p_i$. Gleiche Schlüssel behalten ihre Dateireihenfolge.

Schritt 4: Zuteilung

Setze einen Zeiger $c := a_B$. Für jede Gruppe in der Reihenfolge: runde c auf die nächste Blockgrenze auf, lege dort den Block an und setze c hinter den Block. Passt der Block nicht mehr in B, brich mit Platzmangel ab.

Schritt 5: Freie Bereiche

Zerlege $B \setminus \bigcup_i N_i$ in die minimale Anzahl von Blöcken.

Schritt 6: Ausgabe und Firewall

Gib den Plan als Tabelle, CSV oder JSON aus und erzeuge optional aus den Zugriffspaaren Regeln für `iptables`, `nftables` oder Cisco.

3.2 Bedarf und Präfixlänge

Lemma 3 (Bitlänge und Logarithmus)**.** Für ganze $r \geq 1$ gilt $(r-1).\text{bit_length}() = \lceil \log_2 r \rceil$.

Beweis. Für ganze $m \geq 0$ ist $m.\text{bit_length}() = \min\{k \geq 0 : m < 2^k\}$. Mit $m = r - 1$ folgt $\min\{k \geq 0 : r - 1 < 2^k\} = \min\{k \geq 0 : r \leq 2^k\} = \lceil \log_2 r \rceil$, denn $r \leq 2^k$ ist äquivalent zu $\log_2 r \leq k$. □

Satz 1 (Minimalität des Blocks)**.** Sei $r \geq 1$ und $k = \lceil \log_2 r \rceil$. Dann ist 2^k die kleinste Zweierpotenz, die mindestens r ist, und es gilt $r \leq 2^k < 2r$. Der Block der Präfixlänge $p = w - k$ ist daher der kleinste Block, der r Adressen fasst. Die Berechnung ist genau dann möglich, wenn $r \leq 2^w$ (sonst wäre $p < 0$).

Beweis. Nach Definition gilt $2^k \geq r$, und für $k' < k$ gilt $2^{k'} \leq 2^{k-1} < r$ (denn $k-1 < \log_2 r$). Also ist 2^k die kleinste Zweierpotenz mit $2^k \geq r$. Für $k = 0$ ist $r = 1$ und $2^k = 1 < 2$. Für $k \geq 1$ gilt $2^{k-1} < r$, also $2^k < 2r$. Die Größe eines Blocks der Präfixlänge p ist 2^{w-p}, daher fasst er genau dann r Adressen, wenn $2^{w-p} \geq r$, also $w - p \geq k$, und das kleinste solche $w - p$ ist k. Es ist $p = w - k \geq 0$ genau dann, wenn $k \leq w$, also wenn $r \leq 2^w$. □

Bemerkung 1. Die Präfixlänge wird im Code als $w - (r - 1)$.bit_length() berechnet (Lemma 3). Dadurch entstehen weder Gleitkommafehler noch Logarithmen.

Lemma 4 (Gleitkomma-Aufrundung)**.** Die Version 1.0.0 berechnete $\lceil \gamma h \rceil$ als `math.ceil(round(h * gamma, 9))`. Hat γ höchstens 8 Nachkommastellen und gilt $h\gamma \leq 10^6$, so ist das Ergebnis exakt $\lceil \gamma h \rceil$. Ohne das Runden ist das im Allgemeinen falsch: `math.ceil(100 * 1.1)` ergibt 111 statt 110.

Beweis. Sei $\hat{\gamma}$ die nächste Gleitkommazahl zu γ ($|\hat{\gamma} - \gamma| \leq 2^{-53}\gamma$). Das berechnete Produkt ist $\hat{x} = h\hat{\gamma}(1 + \delta)$ mit $|\delta| \leq 2^{-53}$. Dann ist

$$|\hat{x} - h\gamma| \leq h\hat{\gamma}\,|\delta| + h\,|\hat{\gamma} - \gamma| \leq h\gamma\,(1 + 2^{-53})\,2^{-53} + h\gamma\,2^{-53} < 2.3 \cdot 10^{-16}\,h\gamma \leq 2.3 \cdot 10^{-10}.$$

Fall 1: $h\gamma = m \in \mathbb{N}$. Dann liegt $\hat{x}$ in $(m - 2.3 \cdot 10^{-10},\, m + 2.3 \cdot 10^{-10})$, das Runden auf 9 Nachkommastellen liefert m, und $\lceil m \rceil = m$. *Fall 2:* $h\gamma = N + f$ mit $N \in \mathbb{N}_0$ und $f \notin \mathbb{Z}$. Da γ höchstens 8 Nachkommastellen hat, ist $h\gamma$ ein Vielfaches von 10^{-8}, also $f \in [10^{-8},\, 1 - 10^{-8}]$. Die Abweichung durch Fehler und Runden beträgt höchstens $2.3 \cdot 10^{-10} + 5 \cdot 10^{-10} + 10^{-10} < 10^{-8}$, daher bleibt der gerundete Wert strikt zwischen N und $N + 1$, und die Aufrundung liefert $N + 1 = \lceil h\gamma \rceil$. Für das Gegenbeispiel: In Gleitkommaarithmetik ist $100 \cdot 1.1 = 110.00000000000001$, die Aufrundung ergibt 111. □

Die Einschränkung auf acht Nachkommastellen und $h\gamma \leq 10^6$ ist seit Version 1.1.0 aufgehoben:

Lemma 5 (Exakte Aufrundung)**.** Seien $h \in \mathbb{N}$ und γ ein Dezimalbruch mit höchstens 15 signifikanten Stellen, der als Python-`float` gelesen wird. Das Modul `exact.py` bestimmt $\gamma = p/q$ aus der kürzesten Dezimaldarstellung (`repr`) des `float` und berechnet $\lceil \gamma h \rceil$ als `-((-p*h) // q)`. Das Ergebnis ist für alle h genau $\lceil \gamma h \rceil$, ohne Beschränkung von $h\gamma$.

Beweis. Zwei verschiedene Dezimalbrüche mit höchstens 15 signifikanten Stellen unterscheiden sich relativ um mindestens 10^{-15}, der Rundungsfehler beim Lesen als `float` beträgt höchstens $2^{-53} \approx 1{,}1 \cdot 10^{-16}$. Sie ergeben also verschiedene `float`-Werte. Jede Zeichenkette mit höchstens 15 Stellen, die zum gelesenen `float` gehört, hat daher den Wert γ, insbesondere die kürzeste, die `repr` liefert. Mit $\gamma = p/q$ ist $\lceil \gamma h \rceil = \lceil ph/q \rceil = -\lfloor -ph/q \rfloor$, und `//` rundet in Python nach unten. Alle Schritte rechnen mit ganzen Zahlen und sind exakt. □

3.3 VLAN-Vergabe

Lemma 6 (VLAN-Vergabe)**.** Sei U die Menge der explizit angegebenen VLAN-IDs (paarweise verschieden, nach Schritt 1). Die automatische Vergabe weist den Gruppen ohne explizite ID in Dateireihenfolge die kleinsten noch nicht verwendeten Werte aus $\{10, 20, \ldots, 4090\}$ zu. Dann gilt: (i) alle vergebenen IDs sind paarweise verschieden und liegen in $\{1, \ldots, 4094\}$, (ii) explizite IDs bleiben unverändert, (iii) die Vergabe schlägt genau dann fehl, wenn mehr automatische IDs benötigt werden als $409 - |U \cap \{10, 20, \ldots, 4090\}|$ verfügbar sind, (iv) die Schleife führt insgesamt höchstens 409 Erhöhungen des Kandidaten aus.

Beweis. Die Menge `used` enthält zu Beginn U und wird um jeden vergebenen Wert erweitert. Der Kandidat c startet bei 10 und wird nur erhöht, solange $c \in$ `used`. Jeder zugewiesene Wert ist deshalb nicht in `used`, danach aber schon, also kann er nicht ein zweites Mal vergeben werden und kollidiert nicht mit expliziten IDs: (i), (ii). Die Werte $10, 20, \ldots, 4090$ sind $(4090 - 10)/10 + 1 = 409$ Stück, und c nimmt nur diese Werte an, solange $c \leq 4094$. Da c nie sinkt und jede Erhöhung einen Wert überspringt, kann c höchstens 409 verschiedene Werte durchlaufen: (iv). Findet sich kein freier Wert ≤ 4094 mehr (der nächste Kandidat 4100 überschreitet die Grenze), bricht die Vergabe mit einem Fehler ab: (iii). □

3.4 Reihenfolge und Zuteilung

Sei $\mathrm{sz}(i) = s_i$ die Blockgröße von g_i. Bei `vlsm` ist die Zuteilungsreihenfolge die Sortierung nach (p_i, π_i) lexikographisch aufsteigend. Wegen $p_i = w - k_i$ bedeutet kleineres p_i eine größere Blockgröße, die Blöcke werden also *nach absteigender Größe* angeordnet, bei gleicher Größe nach aufsteigender Priorität, bei weiterer Gleichheit nach Dateireihenfolge (die Sortierung ist stabil). Bei `fixed` ist die Präfixlänge einheitlich $p^* = \min_i p_i$ (größter Block), und es wird nach π_i (stabil) sortiert.

Lemma 7 (Aufrunden auf Blockgrenzen)**.** Für ganze $x \geq 0$ und $m \geq 1$ gilt $-\lfloor -x/m \rfloor = \lceil x/m \rceil$. Der Ausdruck `-(-cursor // size) * size` ist daher das kleinste Vielfache von `size`, das mindestens `cursor` ist.

Beweis. Es ist $\lceil y \rceil = -\lfloor -y \rfloor$ für alle reellen y, und die Ganzzahldivision `//` ist in Python die Abrundung. Das kleinste Vielfache von m mit $\geq x$ ist $m\lceil x/m \rceil$. □

Satz 2 (Lückenlosigkeit)**.** Seien $t_1, t_2, \ldots, t_n$ die Blockgrößen in Zuteilungsreihenfolge und gelte $t_1 \geq t_2 \geq \cdots \geq t_n$ (dies ist bei `vlsm` der Fall, bei `fixed` sind alle t_j gleich). Ist $t_j \leq |B|$ für alle j, so beginnt der j-te Block exakt bei $c_{j-1} = a_B + \sum_{i<j} t_i$, das Aufrunden verändert den Zeiger nie, und die belegten Adressen bilden das lückenlose Intervall $[a_B,\ a_B + \sum_i t_i)$ (soweit der Platz reicht).

Beweis. Induktion über j. Für $j = 1$ ist der Zeiger $c_0 = a_B$ und a_B ist durch $|B| = 2^{w-p_B}$ teilbar, also wegen $t_1 \leq |B|$ (beides Zweierpotenzen) auch durch t_1. Sei die Behauptung für $j - 1$ gezeigt, also $c_{j-1} = a_B + \sum_{i<j} t_i$. Es gilt $t_j \mid |B| \mid a_B$ wegen $t_j \leq |B|$. Für $i < j$ gilt $t_j \mid t_i$, da beide Zweierpotenzen sind und $t_j \leq t_i$. Also teilt t_j die Summe $\sum_{i<j} t_i$ und damit c_{j-1}. Das Aufrunden (Lemma 7) lässt c_{j-1} unverändert, der Block j startet bei c_{j-1} und endet bei $c_{j-1} + t_j - 1$; der neue Zeiger ist $c_j = c_{j-1} + t_j$. □

Satz 3 (Existenz eines Plans)**.** Sei $S' = S = \sum_i s_i$ bei `vlsm` und $S' = n \cdot \max_i s_i$ bei `fixed`. Der Algorithmus bricht genau dann mit einem Platzmangel-Fehler ab, wenn $S' > |B|$. Bei `vlsm` existiert in diesem Fall überhaupt keine überlappungsfreie Anordnung von Blöcken der Größen $s_1, \ldots, s_n$ in B (gleichgültig, mit welchem Verfahren oder in welcher Reihenfolge).

Beweis. Seien $t_1, \ldots, t_n$ die Blockgrößen in Zuteilungsreihenfolge (bei `fixed` alle gleich $\max_i s_i$). *Fall A:* Ein $t_j > |B|$. Dann ist die Präfixlänge kleiner als p_B und der Code löst die Fehlermeldung „benötigt ein /p, das Basisnetz ist kleiner“ aus; zugleich ist $S' \geq t_j > |B|$. *Fall B:* Alle $t_j \leq |B|$. Nach Satz 2 beginnt Block j bei c_{j-1}, und die Prüfung `start + size - 1 > end` bedeutet $c_{j-1} + t_j - 1 > a_B + |B| - 1$, also $\sum_{i \leq j} t_i > |B|$. Da die Partialsummen monoton wachsen, tritt dies für ein j genau dann ein, wenn $\sum_{i \leq n} t_i = S' > |B|$. Dies zeigt die Äquivalenz. Für die letzte Aussage: Paarweise disjunkte Blöcke in B haben zusammen höchstens $|B|$ Adressen, also ist $S > |B|$ unmöglich. □

Satz 4 (Korrektheit der Zuteilung)**.** Bricht der Algorithmus nicht ab, so gilt für das Ergebnis:

(K1) Jedes N_i ist ein ausgerichteter Block der Präfixlänge p_i (bei `fixed`: p^*).

(K2) Es gilt $N_i \subseteq B$ für alle i.

(K3) Die Subnetze sind paarweise disjunkt.

(K4) Jedes Subnetz hat mindestens $\lceil \gamma h_i \rceil$ nutzbare Hosts.

(K5) Die Subnetze werden in streng aufsteigender Adressreihenfolge ausgegeben.

(K6) Bei `vlsm` ist N_i der *kleinste* Block mit (K4), das heißt kein Block mit weniger Adressen erfüllt (K4).

Beweis. (K1) Nach Satz 2 beginnt Block j bei c_{j-1}, das durch t_j teilbar ist; das ist die Ausrichtung. (K2) Es ist $c_{j-1} \geq a_B$, und die Prüfung gegen `end` schließt ein Überschreiten aus. (K3) und (K5) Die Intervalle $[c_{j-1}, c_{j-1} + t_j)$ für $j = 1, \ldots, n$ sind nach Konstruktion hintereinander angeordnet und überlappen nicht. (K4) Nach Lemma 2 gilt $u = s - o \geq \lceil \gamma h_i \rceil$ für $s \geq r_i$; bei `vlsm` ist $s = s_i \geq r_i$, bei `fixed` ist $s = \max_j s_j \geq s_i \geq r_i$. (K6) folgt aus Satz 1: Ein Block mit weniger als s_i Adressen hat höchstens $s_i/2 < r_i$ Adressen, also $u < \lceil \gamma h_i \rceil$ nach Lemma 2. (Genauer: $u = s - o \leq s_i/2 - o < r_i - o = \lceil \gamma h_i \rceil$.) □

Satz 5 (Kompaktheit und Optimalität der Reihenfolge)**.** Bei `vlsm` belegt der Plan genau das Intervall $[a_B,\ a_B + S)$. Jede überlappungsfreie Anordnung von Blöcken der Größen $s_1, \ldots, s_n$ in $\mathcal{A}_w$ mit Anfang bei Adresse $\geq a_B$ hat eine größte belegte Adresse von mindestens $a_B + S - 1$. Die Sortierung nach absteigender Größe ist daher optimal unter *allen* Anordnungen, und nicht nur unter allen Reihenfolgen. Dagegen kann eine andere Reihenfolge strikt mehr Platz benötigen.

Beweis. Die erste Aussage ist Satz 2. Für die zweite: Die Blöcke liegen disjunkt in $[a_B, M]$, wobei M die größte belegte Adresse ist, daher $S \leq M - a_B + 1$. Der VLSM-Plan erreicht $M = a_B + S - 1$. Für die letzte Aussage betrachten wir die Größen 32 und 256 bei $a_B = 0$ in aufsteigender Reihenfolge: Der 32er-Block belegt $[0, 32)$, der 256er-Block muss bei einem Vielfachen von 256 beginnen, also bei 256, und endet bei 511; die Sortierung nach absteigender Größe endet dagegen bei 287. □

Bemerkung 2. Ein naives Verfahren, das alle $n!$ Reihenfolgen ausprobiert, um eine lückenlose Anordnung zu finden, ist wegen Satz 5 überflüssig: Eine einzige Sortierung ist bereits optimal. Das entspricht dem Übergang von $n!$ Permutationen zu n Rotationen beim Subgraph Algorithmus.

Satz 6 (VLSM dominiert feste Präfixlänge)**.** Ist der Plan bei `fixed` möglich, so auch bei `vlsm`. Der Mehrverbrauch von `fixed` gegenüber `vlsm` beträgt $W = n \max_i s_i - \sum_i s_i \geq 0$ Adressen, mit Gleichheit genau dann, wenn alle Blockgrößen gleich sind.

Beweis. Es ist $\sum_i s_i \leq n \max_i s_i$, mit Gleichheit genau dann, wenn alle $s_i = \max_j s_j$. Mit Satz 3 folgt die Behauptung. □

Satz 7 (Interner Verschnitt)**.** Bei `vlsm` gilt $s_i < 2r_i$ für jedes i, die Blockauslastung r_i/s_i liegt also strikt über $1/2$, und die Gesamtbelegung erfüllt $S < 2\sum_i r_i$.

Beweis. Satz 1 liefert $s_i < 2r_i$. Summation über i liefert $S < 2\sum_i r_i$. □

3.5 Freie Bereiche

Seien $e = a_B + |B|$ das Ende des Basisnetzes (exklusiv), $F = |B| - S'$ die Zahl der freien Adressen und $c = e - F$. Bei beiden Strategien liegen die Subnetze lückenlos ab a_B (Satz 2), der freie Bereich ist also das Suffix $[c, e)$. Wir schreiben $F = \sum_{j=1}^{m} 2^{b_j}$ mit $b_1 < \cdots < b_m$ in Binärdarstellung, also $m = \mathrm{popcnt}(F)$.

Satz 8 (Freie Bereiche)**.** Ist $F > 0$, so ist

$$C_j = N(c_j,\ w - b_j) \quad \text{mit} \quad c_j = c + \sum_{i<j} 2^{b_i} = e - \sum_{i \geq j} 2^{b_i}, \qquad j = 1, \ldots, m,$$

eine Zerlegung von $[c, e)$ in ausgerichtete Blöcke mit paarweise verschiedenen Größen $2^{b_1} < \cdots < 2^{b_m}$. Es gilt:

(F1) Jede Zerlegung von $[c, e)$ in Blöcke besteht aus mindestens $\mathrm{popcnt}(F)$ Blöcken, und die Zerlegung (C_j) ist die einzige mit genau $\mathrm{popcnt}(F)$ Blöcken.

(F2) Die Blöcke C_j sind *maximal*: Kein Block, der in $[c, e)$ enthalten ist, enthält ein C_j echt.

(F3) Der größte freie Block hat die Größe $2^{\lfloor \log_2 F \rfloor}$, und es gibt höchstens $w - p_B$ freie Blöcke.

Beweis. Ausrichtung. Da a_B durch $|B|$ teilbar ist, ist es auch e. Wegen $S' \geq 1$ ist $F < |B|$, also $b_m < w - p_B$ und $2^{b_j} \mid |B| \mid e$. Weiter ist $\sum_{i \geq j} 2^{b_i}$ durch 2^{b_j} teilbar, also ist $c_j = e - \sum_{i \geq j} 2^{b_i}$ durch 2^{b_j} teilbar: C_j ist ausgerichtet. Die C_j schließen nach Konstruktion lückenlos aneinander an und überdecken $[c_1, e) = [c, e)$.

(F1) Eine Zerlegung in t Blöcke hat Größen $2^{d_1}, \ldots, 2^{d_t}$ mit Summe F. Gibt es zwei gleiche Summanden 2^d, ersetze sie durch 2^{d+1}; die Summe bleibt F, die Anzahl sinkt um 1. Das Verfahren endet bei paarweise verschiedenen Zweierpotenzen mit Summe F, das ist die eindeutige Binärdarstellung mit $\mathrm{popcnt}(F)$ Summanden. Also $t \geq \mathrm{popcnt}(F)$. Gilt $t = \mathrm{popcnt}(F)$, so wurde nie ersetzt, die Größen sind verschieden und gleich $\{2^{b_j}\}$. Der erste Block beginnt bei c; $c = e - F$ hat dieselbe niedrigste gesetzte Stelle wie F (denn e ist durch $2^{w-p_B} > 2^{b_m}$ teilbar), nämlich 2^{b_1}. Ein bei c beginnender Block muss ausgerichtet

sein, seine Größe muss also c teilen, daher $\le 2^{b_1}$ und folglich $= 2^{b_1}$. Induktiv über den Rest $[c + 2^{b_1}, e)$ mit $F - 2^{b_1}$ ergibt sich die Reihenfolge $b_1 < \cdots < b_m$, also genau (C_j).

(F2) Angenommen, ein Block $M \subseteq [c, e)$ enthalte C_j echt. Dann enthält M auch den Elternblock $\hat{C}_j$ von C_j (den ausgerichteten Block der doppelten Größe, der C_j enthält; nach Lemma 1 ist $\hat{C}_j \subseteq M$). Es ist $c_j \equiv e - 2^{b_j} \equiv 2^{b_j} \pmod{2^{b_j+1}}$, denn e und $\sum_{i>j} 2^{b_i}$ sind durch 2^{b_j+1} teilbar. Also ist c_j *nicht* durch 2^{b_j+1} teilbar, C_j ist die rechte Hälfte seines Elternblocks, und $\hat{C}_j = [c_j - 2^{b_j},\, c_j + 2^{b_j})$. Wegen $\sum_{i<j} 2^{b_i} < 2^{b_j}$ gilt $c_j - 2^{b_j} < c$, also $\hat{C}_j \not\subseteq [c, e)$, ein Widerspruch.

(F3) Der größte Block ist 2^{b_m}, die höchste gesetzte Stelle von F, also $2^{\lfloor \log_2 F \rfloor}$. Wegen $F < |B| = 2^{w-p_B}$ gibt es höchstens $w - p_B$ gesetzte Stellen. □

Bemerkung 3. Die Python-Funktion `ipaddress.summarize_address_range` liefert die minimale CIDR-Zerlegung eines Adressintervalls; nach (F1) ist diese eindeutig und gleich (C_j). In den Zufallstests (Abschnitt 4.6) wurde für alle erzeugten Pläne bestätigt, dass die ausgegebenen freien Bereiche genau den gesetzten Bits von F entsprechen. Die allgemeinere Funktion `_free_ranges` behandelt zusätzlich Lücken vor und zwischen Netzen und wird für die in Abschnitt 7.2 beschriebenen Bestandsnetze benötigt.

Beispiel 3 (Freie Bereiche des Konfigurationsbeispiels)**.** Im Beispiel ist $|B| = 2^{16} = 65\,536$ und $S = 256+128+64+64+32 = 544$, also $F = 64\,992 = 2^{15}+2^{14}+2^{13}+2^{12}+2^{11}+2^{10}+2^{8}+2^{7}+2^{6}+2^{5}$. Das sind $\mathrm{popcnt}(F) = 10$ Blöcke der Größen $32, 64, 128, 256, 1024, \ldots, 32768$, also genau die zehn ausgegebenen Bereiche `10.20.2.32/27` bis `10.20.128.0/17`. Das Fehlen von $2^9 = 512$ erklärt, warum zwischen `/24` und `/22` kein `/23` auftaucht: Das einzige ausgerichtete `/23` an dieser Stelle wäre `10.20.2.0/23`, und dieses enthält den belegten Drucker-Block `10.20.2.0/27`.

Satz 9 (Packbarkeit)**.** Sei $\mathcal{F}$ eine endliche Familie paarweise disjunkter, *maximaler* Blöcke (kein Block in $\bigcup \mathcal{F}$ enthält ein Element von $\mathcal{F}$ echt), und seien $2^{j_1}, \ldots, 2^{j_q}$ Auftragsgrößen. Es gibt genau dann paarweise disjunkte ausgerichtete Blöcke M_t der Größe 2^{j_t} mit $M_t \subseteq \bigcup \mathcal{F}$, wenn für alle $j \ge 0$ gilt

$$\sum_{t:\ j_t \ge j} 2^{j_t} \;\le \sum_{C \in \mathcal{F}:\ |C| \ge 2^j} |C|. \tag{$*$}$$

Beweis. Notwendig. Sei $M_t \subseteq \bigcup \mathcal{F}$ und M_t schneide $C \in \mathcal{F}$. Nach Lemma 1 gilt $M_t \subseteq C$ oder $C \subsetneq M_t$. Der zweite Fall ist wegen der Maximalität ausgeschlossen. Also liegt jedes M_t in einem einzigen C mit $|C| \ge |M_t|$. Die Aufträge mit $j_t \ge j$ liegen damit in Blöcken C mit $|C| \ge 2^j$ und sind dort disjunkt, ihre Gesamtgröße ist also höchstens die Summe der $|C|$ dieser Blöcke.

Hinreichend. Wir platzieren die Aufträge klassenweise für $j = J, J-1, \ldots, 0$ (J die größte vorkommende Größe) und halten die Invariante: *Vor Klasse j ist jeder Block C ab seinem Anfang lückenlos mit bereits platzierten Aufträgen bis zum Füllstand f_C gefüllt, und f_C ist ein Vielfaches von 2^{j+1}.* Anfangs ist $f_C = 0$. Für Blöcke mit $|C| \ge 2^j$ ist die Restkapazität $\rho_C = |C| - f_C$ ein Vielfaches von 2^j (beide Terme sind es). Blöcke mit $|C| < 2^j$ nehmen keine Aufträge der Klassen $\ge j$ auf und bleiben leer. Die gesamte Restkapazität dieser Blöcke ist

$$R_j = \sum_{|C| \ge 2^j} |C| - \sum_C f_C \;\ge \sum_{t:\ j_t \ge j} 2^{j_t} - \sum_{t:\ j_t > j} 2^{j_t} = n_j \cdot 2^j,$$

wobei n_j die Zahl der Aufträge der Größe 2^j ist; dabei wurde $(*)$ und $\sum_C f_C = \sum_{t:\, j_t > j} 2^{j_t}$ benutzt. Da jedes ρ_C ein Vielfaches von 2^j ist, passen in die Blöcke insgesamt $R_j / 2^j \ge n_j$

Aufträge der Größe 2^j. Man platziert sie lückenlos ab dem jeweiligen Füllstand. Die Startadresse $\mathrm{start}(C) + f_C$ ist ein Vielfaches von 2^j (denn $\mathrm{start}(C)$ ist ein Vielfaches von $|C| \geq 2^j$ und f_C ein Vielfaches von 2^{j+1}), die Aufträge sind also ausgerichtet, und die neuen Füllstände sind Vielfache von 2^j, was die Invariante für Klasse $j-1$ herstellt. □

Korollar 1 (Verlustfreie Erweiterbarkeit)**.** Seien nach einem Plan (`vlsm` oder `fixed`) neue Gruppen mit Blockgrößen $x_1, \ldots, x_q$ (Zweierpotenzen) zu ergänzen. Sie lassen sich genau dann in den freien Bereichen unterbringen, ohne bestehende Subnetze zu verschieben, wenn $\sum_t x_t \leq F$. Das ist dieselbe Bedingung, unter der ein vollständig neuer VLSM-Plan aller Gruppen möglich wäre.

Beweis. Die freien Blöcke $C_1, \ldots, C_m$ sind nach Satz 8 (F2) maximal. Notwendigkeit ist klar (die Aufträge liegen disjunkt in F freien Adressen). Für die Hinlänglichkeit prüfen wir $(*)$ für jedes j: Die rechte Seite ist $\sum_{b_i \geq j} 2^{b_i} = F - (F \bmod 2^j)$. Die linke Seite $L_j = \sum_{t:x_t \geq 2^j} x_t$ ist ein Vielfaches von 2^j und höchstens $\sum_t x_t \leq F$, also $L_j \leq \lfloor F/2^j \rfloor\, 2^j = F - (F \bmod 2^j)$. Nach Satz 9 ist die Platzierung möglich. Ein neuer VLSM-Plan aller Gruppen ist nach Satz 3 genau dann möglich, wenn $S + \sum_t x_t \leq |B|$, das heißt $\sum_t x_t \leq F$. □

Bemerkung 4. Der Beweis von Satz 9 ist konstruktiv: Er liefert ein Verfahren „größte Aufträge zuerst, jeweils lückenlos in einen freien Block“, das der Ausblick (Abschnitt 7.2) als inkrementelle Planung vorschlägt.

3.6 Firewall-Regeln

Für ein Zugriffspaar $(i, j) \in P$ (Definition 6) wird eine Freigabe vom Subnetz N_i in das Subnetz N_j erzeugt. Zugriff auf das eigene Subnetz benötigt keine Regel, da Verkehr innerhalb eines Subnetzes (einer Broadcast-Domäne) nicht geroutet wird.

Definition 7 (Regelliste mit Standardaktion)**.** Ein *Paket* ist ein Tripel (x, y, ζ) aus Quelladresse x, Zieladresse y und Zustand $\zeta \in \{\textsc{neu}, \textsc{etabliert}\}$ (ETABLIERT umfasst die Zustände *established* und *related*). Eine *Regelliste* ist eine Folge von Regeln (Bedingung, Aktion) mit einer Standardaktion d. Das *Urteil* ist die Aktion der ersten zutreffenden Regel oder, falls keine zutrifft, d.

Die erzeugte Liste für `iptables` und `nftables` ist: Regel 0: Zustand ETABLIERT $\Rightarrow$ ACCEPT; danach je Zugriffspaar (i, j) die Regel „$x \in N_i \wedge y \in N_j \Rightarrow$ ACCEPT“; Standardaktion DROP (bei `nftables` als `policy drop`, bei `iptables` als letzte Regel `-j DROP`).

Satz 10 (Default-Deny-Korrektheit)**.** Für jedes Paket $(x, y, \textsc{neu})$ ist das Urteil der erzeugten Liste genau dann ACCEPT, wenn es Gruppen $i \neq j$ mit $x \in N_i$, $y \in N_j$ und $g_j \in \mathrm{acc}(g_i)$ gibt. Diese Gruppen sind eindeutig. Pakete mit Quelle oder Ziel außerhalb aller Subnetze (zum Beispiel ins Internet) werden verworfen. Die Reihenfolge der Freigabe-Regeln beeinflusst das Urteil nicht.

Beweis. Regel 0 trifft auf NEU nicht zu. Eine Freigabe-Regel für $(i, j) \in P$ trifft genau dann zu, wenn $x \in N_i$ und $y \in N_j$. Existiert keine zutreffende Freigabe, ist das Urteil DROP. Nach Satz 4 (K3) sind die N_i disjunkt, jede Adresse liegt also in höchstens einem N_i: Die Gruppen i und j sind eindeutig, falls sie existieren, und dann ist $(i, j) \in P$ genau dann, wenn $i \neq j$ und $g_j \in \mathrm{acc}(g_i)$. Liegt x oder y in keinem Subnetz, trifft keine Freigabe zu. Da alle Freigaben dieselbe Aktion ACCEPT haben, ist das Urteil von ihrer Reihenfolge unabhängig. □

Korollar 2 (Richtung und Rückverkehr)**.** Zugriff ist gerichtet und nicht transitiv: Dass g_i auf g_j und g_j auf g_k zugreifen darf, erlaubt nicht automatisch $g_i \to g_k$. Ein neues Paket von g_j nach g_i wird nur freigegeben, wenn $(j, i) \in P$. Antworten auf bereits aufgebaute Verbindungen sind dagegen als ETABLIERT immer zulässig. Die Zahl der Freigabe-Regeln ist $|P| \leq n(n-1)$, bei doppelt genannten Zugriffszielen in `access` entsprechend mehr identische Regeln.

Beweis. Folgt direkt aus Satz 10 und der Definition von P. Die Schranke $|P| \leq n(n-1)$ gilt, weil $P \subseteq \{(i, j) : i \neq j\}$ ist, solange jedes Ziel höchstens einmal genannt wird. □

Cisco-ACLs. Für jedes Subnetz N_i wird eine eingehende ACL am Interface `Vlan<ID>` erzeugt, die für jedes $(i, j) \in P$ die Zeile `permit ip` N_i N_j in Wildcard-Schreibweise (Hostmaske) und am Ende `deny ip any any` enthält.

Satz 11 (Cisco-Semantik)**.** Ein Paket (x, y), das am Interface `Vlan`v_i eingeht, wird genau dann weitergeleitet, wenn $x \in N_i$ und es ein $j \neq i$ mit $g_j \in \mathrm{acc}(g_i)$ und $y \in N_j$ gibt. Insbesondere werden Pakete mit gefälschter Quelle (außerhalb N_i) verworfen. Da ACLs zustandslos sind, wird Rückverkehr von g_j nach g_i nur dann weitergeleitet, wenn zusätzlich $g_i \in \mathrm{acc}(g_j)$ gilt.

Beweis. Die ACL besteht aus den `permit`-Zeilen für $(i, j) \in P$ mit Quelle N_i und Ziel N_j und der Schlusszeile `deny`; die erste zutreffende Zeile entscheidet. Ein Paket wird also genau dann erlaubt, wenn $x \in N_i \wedge y \in N_j$ für ein $(i, j) \in P$. Die Aussage über Fälschungen folgt aus $x \notin N_i$. Ein Rückpaket von g_j nach g_i tritt an Vlan v_j ein und unterliegt der ACL von g_j, die es nur bei $(j, i) \in P$ erlaubt. □

Bemerkung 5 (Konsequenz für das Konfigurationsbeispiel)**.** Im Beispiel gibt es kein symmetrisches Paar. Die erzeugten Cisco-ACLs verhindern daher, dass etwa Antworten von `Server` an `Entwicklung` zurückkehren, und die Verbindung kommt nicht zustande. Die Kopfzeile der Ausgabe weist darauf hin, dass der Rückverkehr durch Gegenregeln zu erlauben ist. Eine automatische Lösung ist im Ausblick beschrieben. Bei `iptables` und `nftables` tritt das Problem wegen der Regel 0 (ETABLIERT) nicht auf.

Lemma 8 (Eindeutige ACL-Namen)**.** Die ACL-Namen `ACL-<VLAN>-<NAME>` sind für verschiedene Gruppen verschieden.

Beweis. Nach Lemma 6 sind die VLAN-IDs paarweise verschieden. Der Name beginnt mit `ACL-`, es folgt die Dezimaldarstellung der VLAN-ID und ein `-`. Da die Dezimaldarstellung nur Ziffern enthält, lässt sich die ID aus dem Namen eindeutig zurücklesen. Verschiedene IDs ergeben also verschiedene Namen. □

3.7 Implementierung

Die Implementierung in Python gliedert sich seit Version 1.1.0 in siebzehn Module. Der Kern aus Abschnitt 3 liegt in `models.py` (Konstanten, Datenklassen, Fehlertyp `PlanError`), `config.py` (Laden und Validieren), `planner.py` (Bedarf, VLAN-Vergabe, Zuteilung), `render.py` (Tabelle, CSV, JSON), `firewall.py` (Regeln) und `cli.py` (Kommandozeile). Die Module der Erweiterungen beschreibt der letzte Teil dieses Abschnitts. Verwendet werden die Standardbibliothek (`ipaddress`, `fractions`, `csv`, `json`), `PyYAML`, `NumPy` für die Zugriffsmatrix und der Subgraph Algorithmus als Abhängigkeit (`pip install` über

die `pyproject.toml`, nicht als Kopie im Projekt). Nur `policy.py` importiert ihn, und die Kommandozeile lädt dieses Modul erst bei `-compare`. Alle Datenklassen sind unveränderlich (`frozen=True`), und alle Fehler werden als `PlanError` gemeldet, die die Kommandozeile mit dem Rückgabewert 1 und der Meldung „Fehler: …“ auf der Fehlerausgabe beantwortet.

3.8 Bedarf und Präfixlänge

Listing 3.1: Bedarf und Präfixlänge (`planner.py`)

```
def default_overhead(gateway_position: str, version: int) -> int:
    """Overhead␣o:␣Netzadresse,␣Broadcast␣(nur␣IPv4)␣und␣Gateway␣(außer␣bei␣``none``)."""
    overhead = 2 if version == 4 else 1  # IPv6 kennt keinen Broadcast
    if gateway_position != "none":
        overhead += 1
    return overhead


def group_overhead(group: Group, gateway_position: str, version: int) -> int:
    """Overhead␣der␣Gruppe:␣ihr␣eigener␣Wert␣(Satz␣"Robustheit")␣oder␣der␣Standard."""
    if group.overhead is not None:
        return group.overhead
    return default_overhead(gateway_position, version)


def required_addresses(group: Group, growth_factor: float, gateway_position: str, version: int) -> int:
    """Adressen␣r␣=␣ceil(gamma␣*␣h)␣+␣o,␣mit␣exakter␣Rechnung␣(siehe␣``exact``)."""
    return ceil_product(group.hosts, growth_factor) + group_overhead(group, gateway_position, version)


def prefix_for(total: int, max_prefix: int, name: str) -> int:
    """Kleinste␣Präfixlänge,␣deren␣Block␣mindestens␣``total``␣Adressen␣fasst."""
    prefix = max_prefix - (total - 1).bit_length()
    if prefix < 0:
        raise PlanError(f"Gruppe␣{name!r}:␣{total}␣Adressen␣passen␣in␣keinen␣Adressraum")
    return prefix
```

Die Funktion `required_addresses` berechnet $r_i = \lceil \gamma h_i \rceil + o_i$ nach Definition 3; `ceil_product` aus `exact.py` rechnet exakt (Lemma 5) und vermeidet damit den Gleitkommafehler aus Lemma 4. Der Overhead o_i ist der Standardwert oder der in der Gruppe gesetzte Wert (Satz 18). Die Funktion `prefix_for` berechnet $p = w - \lceil \log_2 r \rceil$ mit der Bitlänge aus Lemma 3 und erfüllt Satz 1.

Beispiel 4. Für die Gruppe `Entwicklung` ist $h = 120$, $\gamma = 1{,}25$, $o = 3$. Es ist $r = 150 + 3 = 153$, $(r-1).\text{bit_length}() = 152.\text{bit_length}() = 8$, denn $152 = 10011000_2$ hat acht Stellen. Also $p = 32 - 8 = 24$.

3.9 VLAN-Vergabe

Listing 3.2: Automatische VLAN-IDs (`planner.py`)

```
def assign_vlans(groups: Sequence[Group]) -> Dict[str, int]:
    """Explizite VLAN-IDs übernehmen, fehlende automatisch
       vergeben (10, 20, ...)."""
    used = {g.vlan_id for g in groups if g.vlan_id is not None}
    result: Dict[str, int] = {}
    candidate = AUTO_VLAN_START
    for g in groups:
        if g.vlan_id is not None:
            result[g.name] = g.vlan_id
            continue
        while candidate in used:
            candidate += AUTO_VLAN_STEP
        if candidate > VLAN_MAX:
            raise PlanError("Keine freie VLAN-ID mehr verfügbar")
        used.add(candidate)
        result[g.name] = candidate
    return result
```

Die Menge `used` enthält von Anfang an alle expliziten IDs. Dadurch kann eine automatisch vergebene ID nie mit einer expliziten kollidieren, auch wenn diese erst später in der Liste steht (Lemma 6).

3.10 Zuteilung der Subnetze

Listing 3.3: Kern der Zuteilung (`planner.py`, Auszug)

```
    if cfg.allocation_strategy == "fixed":
        uniform = min(prefix for _, prefix in sized)  # kleinste
           Länge = größter Block
        sized = [(g, uniform) for g, _ in sized]
    order = _ordered(sized, cfg.allocation_strategy)

    vlans = assign_vlans(cfg.groups)
    net_cls = type(base)
    cursor = int(base.network_address)
    end = int(base.broadcast_address)
    subnets: List[Subnet] = []

    for group, prefix in order:
        if prefix < base.prefixlen:
            raise PlanError(
```

```
                f"Platzmangel: Gruppe {group.name!r} benötigt ein /{prefix}, "
                f"das Basisnetz {base} ist kleiner"
            )
        size = 1 << (max_prefix - prefix)
        start = -(-cursor // size) * size  # auf Blockgrenze aufrunden
        if start + size - 1 > end:
            raise PlanError(
                f"Platzmangel: Gruppe {group.name!r} (/{prefix}) passt nicht mehr in {base}"
            )
        net = net_cls((start, prefix))
        subnets.append(_build_subnet(group, vlans[group.name], net, cfg.gateway_position))
        cursor = start + size
```

Der Code entspricht Schritt 4. Die Hilfsfunktion `_ordered` sortiert bei `fixed` nach der Priorität und bei `vlsm` nach (Präfixlänge, Priorität). Python-Sortierung ist stabil, daher entscheidet bei gleichem Schlüssel die Dateireihenfolge. Die Variable `cursor` ist der Zeiger c. Die Zeile `start = -(-cursor // size) * size` ist das Aufrunden aus Lemma 7; bei `vlsm` ändert sie den Zeiger nie (Satz 2), sie ist also eine Sicherung und kein Platzverlust. Die beiden `raise`-Anweisungen sind genau die Fälle A und B aus dem Beweis von Satz 3.

3.11 Freie Bereiche

Listing 3.4: Freie Restbereiche (`blocks.py`)

```
def free_ranges_in_pool(pool: IPNetwork, occupied: Sequence[IPNetwork]) -> Tuple[IPNetwork, ...]:
    addr = type(pool.network_address)
    pos = int(pool.network_address)
    end = int(pool.broadcast_address)
    free: List[IPNetwork] = []
    for net in sorted(occupied, key=lambda n: int(n.network_address)):
        start = int(net.network_address)
        if start > pos:
            free.extend(ipaddress.summarize_address_range(addr(pos), addr(start - 1)))
        pos = int(net.broadcast_address) + 1
    if pos <= end:
        free.extend(ipaddress.summarize_address_range(addr(pos), addr(end)))
    return tuple(free)
```

Die Funktion liegt seit Version 1.1.0 in `blocks.py`; `planner._free_ranges` ruft sie auf. Sie läuft in Adressreihenfolge über die belegten Netze und fasst jede Lücke mit `summarize_address_range` zu minimal vielen Blöcken zusammen. Für die Pläne des Algorithmus gibt es nur die Lücke hinter dem letzten Netz (Abschnitt 3.5); die Funktion ist allgemeiner und wird in den Tests auch mit Lücken vor und zwischen Netzen geprüft.

3.12 Firewall-Regeln

Listing 3.5: Zugriffspaare und nftables-Ausgabe (`firewall.py`)

```
def _access_pairs(plan: Plan) -> Iterator[Tuple[Subnet, Subnet]]:
    """(Quelle, Ziel)-Paare; Zugriff auf das eigene Netz braucht keine Regel."""
    by_name = {s.group: s for s in plan.subnets}
    for source in plan.subnets:
        for target in source.access:
            if target != source.group:
                yield source, by_name[target]


def render_nftables(plan: Plan) -> str:
    family = "ip" if plan.base_network.version == 4 else "ip6"
    lines = [
        "# Generiert von subnetplan: Standard ist Verwerfen, erlaubt sind nur die Gruppenzugriffe.",
        "table inet subnet_acl",
        "delete table inet subnet_acl",
        "table inet subnet_acl {",
        "    chain forward {",
        "        type filter hook forward priority 0; policy drop;",
        "        ct state established,related accept",
    ]
    for src, dst in _access_pairs(plan):
        lines.append(
            f'        {family} saddr {src.network} {family} daddr {dst.network} '
            f'accept comment "{src.group} -> {dst.group}"'
        )
    lines += ["    }", "}"]
    return "\n".join(lines)
```

Der Generator `_access_pairs` erzeugt genau die Menge P aus Definition 6 (mit Wiederholungen, falls ein Ziel mehrfach genannt wird). Die ersten beiden Zeilen der `nftables`-Ausgabe, `table inet subnet_acl` und `delete table inet subnet_acl`, bilden einen bewährten Kniff: Die erste Zeile legt die Tabelle an, falls sie fehlt, die zweite löscht sie, sodass die Datei beim wiederholten Einspielen kein Fehler erzeugt und stets den kompletten Stand ersetzt. Die Kette hat die Standardaktion `policy drop`, die Regel `ct state established,related accept` entspricht der Regel 0 aus Abschnitt 3.6.

3.13 Validierung der Konfiguration

Listing 3.6: Konsistenzprüfung der Gruppen (`config.py`)

```
def _check_groups(groups: Sequence[Group]) -> None:
    """Validierung in O(n + a) mit Zählern statt ``list.count``."""
```

```
    name_counts = Counter(g.name for g in groups)
    duplicates = sorted(n for n, c in name_counts.items() if c > 1)
    if duplicates:
        raise PlanError(f"Doppelte Gruppennamen: {duplicates}")

    vlan_counts = Counter(g.vlan_id for g in groups if g.vlan_id is not None)
    dup_vlans = sorted(v for v, c in vlan_counts.items() if c > 1)
    if dup_vlans:
        raise PlanError(f"Doppelte VLAN-IDs: {dup_vlans}")

    known = set(name_counts)
    for g in groups:
        missing = sorted(set(g.access) - known)
        if missing:
            raise PlanError(f"Gruppe {g.name!r}: access verweist auf unbekannte Gruppen {missing}")

    with_site = [g for g in groups if g.site is not None]
    if with_site and len(with_site) != len(groups):
        raise PlanError("'site' muss entweder für alle Gruppen oder für keine angegeben werden")
```

Die Prüfung benötigt dank der Zähler nur $O(n + a)$ statt $O(n^2)$ Zeit. Anschließend entfernt `clean_access` Wiederholungen und Selbstbezüge (Satz 15). Die Funktion `_check_groups` stellt sicher, dass die Voraussetzungen von Lemma 6 (paarweise verschiedene explizite VLAN-IDs) und von Satz 10 (jedes Zugriffsziel ist eine existierende Gruppe, sodass `by_name[target]` nie fehlschlägt) erfüllt sind. Die Namensprüfung mit dem regulären Ausdruck `^[\w\- ]+$` verhindert, dass Sonderzeichen in Firewall-Kommentare oder ACL-Namen gelangen.

3.14 Module der Erweiterungen

Die Erweiterungen aus Abschnitt 7.2 liegen in eigenen Modulen, sodass der Kern aus diesem Abschnitt unverändert bleibt. Für Pläne mit einem Pool und ohne Bestand läuft weiterhin genau der Algorithmus aus Abschnitt 3 (`_plan_simple`); nur bei `existing`, `pools` oder festen Netzen (`network`) übernimmt `_plan_general`, bei Standorten (`site`) `plan_hierarchical`. So bleiben die Sätze 2 und 5 für den Normalfall gültig.

Modul	Aufgabe
`exact.py`	exakte Bedarfsrechnung $\lceil \gamma h \rceil$ (Lemma 5)
`blocks.py`	freie Bereiche, Bedingung $(*)$, konstruktive Zuteilung (Sätze 9, 17)
`access.py`	Zugriffsmatrix, `clean_access` (Satz 15), `rule_groups` (Satz 16)
`semantics.py`	Referenzurteile der Regelwerke (Sätze 10, 11)
`hierarchy.py`	Standortblöcke und Gruppen darin (Satz 19)
`extend.py`	Plan einlesen, erweitern, verdoppeln (Korollar 1, Satz 21)
`roundtrip.py`	Regeln zurücklesen und mit P vergleichen
`verify.py`	unabhängige Prüfung eines Plans gegen die Sätze
`stats.py`	Auslastung, Verschnitt, Poisson-Dimensionierung (Satz 20)
`checks.py`	Warnungen (Adressbereich, VPN-Überschneidung, Auslastung)
`policy.py`	Richtlinienvergleich mit dem Subgraph Algorithmus (Abschnitt 7.7)

Die Firewall-Optionen sind in der Datenklasse `FirewallOptions` gebündelt (`idempotent`, `group_rules`, `cisco_return`, `ascii_names`, `log_drops`, `icmpv6_types`); die Kommandozeile bildet sie auf `-no-idempotent`, `-group-rules`, `-cisco-return`, `-ascii-names`, `-log-drops` und `-icmpv6` ab. Berichte (`-verify`, `-roundtrip`, `-warnings`, `-stats`, `-compare`) gehen auf die Fehlerausgabe, damit die Standardausgabe maschinenlesbar bleibt; eine fehlgeschlagene Prüfung ergibt den Rückgabewert 1.

Kapitel 4

Analyse

4.1 Laufzeit

Im Folgenden sei n die Zahl der Gruppen und a die Gesamtzahl der Einträge in allen `access`-Listen. Operationen auf ganzen Zahlen der Breite $w \leq 128$ zählen als Einheitskosten.

Satz 12 (Laufzeit der Implementierung)**.** Die Planung (Schritte 1 bis 5) hat die Laufzeit $O(n^2 + a)$, die Ausgabe als Tabelle, CSV oder JSON $O(n + a)$, die Firewall-Ausgabe für `iptables` und `nftables` $O(n + a)$ und für Cisco $O(n\,(n + a))$. Der Kern (Bedarf, Sortierung, Zuteilung, Freibereiche) benötigt $O(n \log n + w)$.

Beweis. **Schritt 1 (Validierung).** Jede Gruppe wird einmal geprüft ($O(n)$ plus $O(|\text{acc}|)$ je Gruppe, zusammen $O(n+a)$). In `_check_groups` wird für jedes Element `names.count(n)` beziehungsweise `vlans.count(v)` aufgerufen, und jeder Aufruf durchläuft die Liste einmal: $O(n)\cdot O(n) = O(n^2)$. Der Zugriffs-Test `set(g.access) - known` kostet $O(|\text{acc}|)$ je Gruppe, zusammen $O(a)$. Gesamt $O(n^2 + a)$.

Schritt 2. Je Gruppe Einheitskosten: $O(n)$.

Schritt 3. Sortieren: $O(n \log n)$. `assign_vlans`: $O(n)$ plus höchstens 409 Erhöhungen (Lemma 6), also $O(n)$.

Schritt 4. Eine Schleife mit n Durchläufen, je $O(1)$ (die Konstruktion eines Subnetzes ist Einheitskosten): $O(n)$.

Schritt 5. Es gibt höchstens $O(n)$ Sortier- und Schleifenschritte für die Netze und höchstens $w - p_B$ Blöcke (Satz 8), also $O(n \log n + w)$.

Die Summe ergibt $O(n^2 + a)$ für die Planung und $O(n \log n + w)$ für den Kern.

Ausgabe. Jedes Subnetz erzeugt eine Zeile mit konstant vielen Feldern, die Zugriffsliste eine Zeichenkette der Länge $O(|\text{acc}|)$: $O(n + a)$.

Firewall. `_access_pairs` durchläuft alle Subnetze und deren Ziele, $O(n + a)$. Bei `iptables` und `nftables` wird der Generator einmal durchlaufen. Bei Cisco wird er für jedes der n Subnetze erneut vollständig durchlaufen (`for src in plan.subnets:` um `for source, dst in _access_pairs(plan)`), das sind $n \cdot O(n + a) = O(n(n + a))$. □

Bemerkung 6. Bei `access`-Listen ohne Wiederholungen ist $a \leq n(n-1)$. Im ungünstigsten Fall ist $a = \Theta(n^2)$, und die Cisco-Ausgabe ist dann kubisch, obwohl ihre Ausgabegröße nur $\Theta(n + a)$ beträgt. Beide quadratischen Stellen (Validierung und Cisco) sind leicht zu beseitigen (Abschnitt 7.2).

4.2 Optimalität des Kerns

Wir zeigen, dass die Sortierung im Kern nicht vermeidbar ist und die Größe der Ausgabe eine weitere untere Schranke ist. Dazu betrachten wir das *Vergleichsmodell*: Die Prioritäten π_i dürfen nur durch Vergleiche (kleiner, gleich, größer) benutzt werden.

Lemma 9 (Untere Schranke für das Sortieren)**.** Jeder Vergleichsalgorithmus, der n paarweise verschiedene Zahlen sortiert, benötigt im schlimmsten Fall mindestens $\lceil \log_2 n! \rceil \geq \frac{n}{2} \log_2 \frac{n}{2}$ Vergleiche.

Beweis. Ein Vergleichsalgorithmus entspricht einem Binärbaum; jedes Blatt gehört zu einer Ausgabepermutation. Da alle $n!$ Permutationen vorkommen müssen, hat der Baum mindestens $n!$ Blätter und damit mindestens die Tiefe $\lceil \log_2 n! \rceil$. Es ist $n! \geq (n/2)^{n/2}$, denn mindestens $n/2$ der Faktoren $1, \ldots, n$ sind $\geq n/2$. Also $\log_2 n! \geq \frac{n}{2} \log_2 \frac{n}{2}$. □

Satz 13 (Optimalität des Kerns)**.** Im Vergleichsmodell benötigt jeder korrekte Planungsalgorithmus $\Omega(n \log n)$ Vergleiche im schlimmsten Fall, und seine Ausgabe hat die Größe $\Omega(n + |P|)$, erfordert also mindestens diese Zeit. Der Kern des Algorithmus hat die Laufzeit $O(n \log n + w)$, der Algorithmus ist damit bis auf den konstanten Faktor w *optimal*. Mit Hash-Tabellen statt Listenzählung wird auch die Validierung zu $O(n + a)$, und die Gesamtlaufzeit ist $\Theta(n \log n + a)$.

Beweis. Untere Schranke $\Omega(n \log n)$. Wähle Gruppen mit gleichem $h_i = 1$, die Strategie `fixed`, ein ausreichend großes Basisnetz und paarweise verschiedene Prioritäten $\pi_1, \ldots, \pi_n$. Alle Blöcke haben dieselbe Größe, die Reihenfolge der Ausgabe in Adressreihenfolge ist nach Satz 4 (K5) genau die Reihenfolge der aufsteigenden Prioritäten. Ein Planungsalgorithmus, der diese Ausgabe liefert, sortiert also die Prioritäten. Nach Lemma 9 sind $\Omega(n \log n)$ Vergleiche nötig.

Ausgabegröße. Der Plan enthält n Subnetze und, wenn Firewall-Regeln gewünscht sind, $|P|$ Freigaben. Eine Ausgabe der Größe $\Omega(n + |P|)$ erfordert mindestens ebenso viele Schritte.

Obere Schranke. Satz 12 liefert $O(n \log n + w)$ für den Kern. Ersetzt man die Listenzählungen in `_check_groups` durch ein Wörterbuch (erwartet $O(1)$ je Zugriff), so ist die Validierung $O(n + a)$; die Cisco-Ausgabe kann mit einer Gruppierung der Paare nach Quelle in $O(n + a)$ erzeugt werden. Die Summe ist $O(n \log n + a)$. □

Bemerkung 7. Die Schranke ist modellabhängig. Sind die Prioritäten ganze Zahlen aus $\{0, \ldots, \Pi\}$, so sortiert Zählsortierung in $O(n + \Pi)$, und die untere Schranke $\Omega(n \log n)$ des Vergleichsmodells gilt nicht. Unbedingt (ohne Modellannahme) gilt nur die lineare Schranke durch die Ausgabegröße.

4.3 Speicher

Die Konfiguration benötigt $O(n+a)$ Speicher, der Plan $O(n+\min(w, \mathrm{popcnt}(F)))$, nämlich n Subnetze und höchstens $w - p_B$ freie Blöcke (Satz 8). Bei einem IPv6-Basisnetz `/48` sind das bis zu 80 freie Blöcke, bei IPv4 `/16` höchstens 16.

4.4 Alternative: Buddy-System

Eine Alternative zur sortierten Zuteilung ist ein binärer Baum über dem Adressraum (ein *Buddy-System*): Jede Anforderung sucht den kleinsten freien Block ausreichender Größe, teilt ihn bei Bedarf und gibt Blöcke wieder zusammen frei. Jede Operation kostet $O(w)$, und das Verfahren unterstützt Anlegen und Löschen im laufenden Betrieb. Für die einmalige Planung aller Gruppen ist die Sortierung mit $O(n \log n)$ einfacher und nach Satz 5 bereits optimal. Das Buddy-System bietet sich für die in Abschnitt 7.2 beschriebene inkrementelle Planung an; der Beweis von Satz 9 zeigt, dass das Verfahren „größte Anforderung zuerst“ dabei ohne Kapazitätsverlust arbeitet.

4.5 Gesamtkorrektheit

Satz 14 (Korrektheit)**.** Zu jeder gültigen Konfiguration terminiert der Algorithmus und liefert entweder

(a) einen Plan, der (K1) bis (K6) aus Satz 4 erfüllt, dessen VLAN-IDs paarweise verschieden sind (Lemma 6), dessen freie Bereiche die minimale Zerlegung von $B \setminus \bigcup_i N_i$ bilden (Satz 8) und dessen Firewall-Regeln Satz 10 erfüllen, oder

(b) eine Fehlermeldung zum Platzmangel, und dann gilt $S' > |B|$ (Satz 3). Bei `vlsm` gibt es in diesem Fall keinen Plan irgendeiner Art.

Beweis. Terminierung. Alle Schleifen laufen über endliche Listen, die einzige `while`-Schleife in `assign_vlans` erhöht den Kandidaten höchstens 409-mal (Lemma 6 (iv)). *Ergebnis.* Tritt kein Fehler auf, gelten (K1) bis (K6) nach Satz 4, die VLAN-Aussage nach Lemma 6, die Aussage über freie Bereiche nach Satz 8 und die Firewall-Aussage nach Satz 10. Tritt ein Platzmangel-Fehler auf, folgt die Aussage aus Satz 3. □

4.6 Experimentelle Überprüfung

Die Testsuite der Version 1.0.0 umfasste 127 Tests und erreichte laut Bericht 100 % Zeilen- und Zweigabdeckung für alle Module (`cli.py`: 54 Anweisungen, `config.py`: 96, `firewall.py`: 47, `models.py`: 68, `planner.py`: 84, `render.py`: 35; zusammen 384 Anweisungen und 108 Zweige). Version 1.1.0 umfasst 247 Tests; neu sind Testdateien für Blockalgebra, Zugriff und Semantik, Bestandsplanung, Hierarchie und Erweiterung, Firewall-Optionen mit Rückübersetzung, Prüfung, Statistik und Warnungen, Richtlinienvergleich, die erweiterte Kommandozeile sowie seeded Zufallstests der Sätze (`tests/test_properties.py`). Eine Ersatzmessung ohne `coverage.py` (Zeilen- und Verzweigungsaufzeichnung mit `sys.settrace`, Anweisungen mit `## pragma: no cover` ausgenommen) fand alle 1 652 ausführbaren Anweisungen ausgeführt und jede verzweigende Anweisung in beiden Richtungen durchlaufen; maßgeblich bleibt der Bericht von `pytest-cov`. Einzelne Tests prüfen unmittelbar die Aussagen dieser Arbeit:

Listing 4.1: Test zu (K1) bis (K3) und Satz 8 (`test_planner.py`)

```
def test_networks_are_aligned_and_do_not_overlap():
    sizes = [3, 200, 17, 60, 9, 1000, 31]
    groups = [{"name": f"G{i}", "hosts": h} for i, h in enumerate
        (sizes)]
```

```
    plan = make_plan(base_network="10.0.0.0/20", groups=groups)
    nets = [s.network for s in plan.subnets]
    for i, net in enumerate(nets):
        assert net.subnet_of(plan.base_network)
        assert all(not net.overlaps(other) for other in nets[i + 1:])
    covered = sum(n.num_addresses for n in nets) + sum(n.num_addresses for n in plan.free_ranges)
    assert covered == plan.total_addresses
```

Listing 4.2: Test zum binären Rest (Satz 8) (`test_planner.py`)

```
def test_single_group_leaves_binary_remainder_as_free_ranges():
    plan = make_plan(groups=[{"name": "G", "hosts": 10}])  # 13 + 3 = 16 -> /28
    assert [str(n) for n in plan.free_ranges] == [
        "10.0.0.16/28",
        "10.0.0.32/27",
        "10.0.0.64/26",
        "10.0.0.128/25",
    ]
```

Der zweite Test ist ein Spezialfall von Satz 8: Im `/24` sind 16 Adressen belegt, $F = 240 = 128 + 64 + 32 + 16$, und die freien Blöcke sind genau die Zweierpotenzen der Binärdarstellung in aufsteigender Reihenfolge.

Vollständige Testabdeckung beweist allerdings nicht die Korrektheit: Sie zeigt, dass jede Zeile und jeder Zweig ausgeführt wurde, nicht dass für *alle* Eingaben die richtigen Werte entstehen. Deshalb wurden die Sätze dieser Arbeit zusätzlich mit einem eigenen Zufallstest gegen die unveränderte Implementierung geprüft (Listing 4.3, gekürzt). Er erzeugte 20 000 Konfigurationen (IPv4 und IPv6, beide Strategien, alle Gateway-Positionen, Wachstumsfaktoren 1,0 bis 2,0, 1 bis 12 Gruppen). Davon waren 12 835 planbar und 7 165 endeten mit Platzmangel; in allen Fällen stimmte das Ergebnis mit den Sätzen überein:

- Planbarkeit genau dann, wenn $S' \le |B|$ (Satz 3).
- Alle Netze ausgerichtet, in B enthalten, paarweise disjunkt; lückenlose Belegung ab a_B (Sätze 4, 2, 5).
- $u = s - o$ (Lemma 2) und $s < 2r$ bei `vlsm` (Satz 7).
- Die freien Bereiche haben genau die Größen der gesetzten Bits von F, in aufsteigender Reihenfolge (Satz 8).
- Die Formel $p = w - \lceil \log_2 r \rceil$ für alle $r \in \{1, \dots, 19\,999\}$ (Satz 1).

Die Packbarkeitsbedingung $(*)$ aus Satz 9 wurde in einem Universum von $2^5 = 32$ Adressen für 4 000 zufällige Belegungen und Aufträge gegen eine vollständige Suche (Backtracking) verglichen; beide Verfahren lieferten stets dasselbe Ergebnis. Schließlich wurde Lemma 4 an 300 000 Zufallspaaren (h, γ) mit $h \le 2 \cdot 10^6$ und zwei Nachkommastellen geprüft (keine Abweichung zur exakten Bruchrechnung).

Listing 4.3: Zufallstest der Sätze (gekürzt)

```
for it in range(20000):
    cfg = rnd_cfg(strategy, gw, v6)              # Zufallskonfiguration
    sizes = [block_size(g, cfg) for g in cfg.groups]
    n = len(sizes)
    total = n * max(sizes) if strategy == "fixed" else sum(sizes)
    feasible = total <= cfg.base_network.num_addresses
    try:
        plan = plan_subnets(cfg)
    except PlanError:
        assert not feasible                      # Satz Existenz
        continue
    assert feasible
    nets = [s.network for s in plan.subnets]
    pos = int(cfg.base_network.network_address)
    for n_ in nets:                              # Lückenlosigkeit
        assert int(n_.network_address) == pos
        pos += n_.num_addresses
    F = cfg.base_network.num_addresses - (pos - int(cfg.base_network.network_address))
    assert sorted(f.num_addresses for f in plan.free_ranges) == \
           [1 << b for b in range(F.bit_length()) if F >> b & 1]
```

Prüfung der Erweiterungen. Das Skript `science/pruefe-erweiterungen.py` prüft die Aussagen aus Abschnitt 7.2 gegen die Implementierung (feste Zufallsstarts, Aufruf `PYTHONPATH=. python science/pruefe-erweiterungen.py`):

Aussage	Prüfung	Ergebnis
Satz 9	Bedingung $(*)$ gegen vollständige Suche in 32 Adressen	4000/4000
Satz 18	Pläne mit Mindestgröße und eigenem Overhead (IPv4, IPv6) erfüllen alle Eigenschaften	2000/2000
Satz 16	gleiche Paarmenge und $\lvert\mathcal{T}\rvert \le n$	2000/2000
Satz 15	genau $\lvert P\rvert$ Regeln nach der Bereinigung	2000/2000
Satz 19	$E_\sigma < 2D_\sigma$ und $\sum E_\sigma < 2\sum s_i < 4\sum r_i$	1000/1000
Rückübersetzung	$P' = P$ (drei Plattformen, zwei Optionssätze, 300 Pläne)	1800/1800
Korollar 1	Erweiterung gelingt genau dann, wenn die Summe $\le F$	1500/1500
Satz 21	Verdopplung korrekt und ohne Umnummerierung (Gateway `first`/`none`)	234/234
Satz 20	Monte-Carlo-Mittel bei 400 000 Stichproben	0,7210 (Soll 0,7213)
Lemma 10	$P \subseteq P'$ gegen Paketurteile an Stichprobenadressen	1500/1500

Beim Schreiben der Tests fiel ein Fehler auf, den die Beweise nicht ausschließen: Feste Netze (`network`) wurden zunächst nicht darauf geprüft, ob sie den Bedarf der Gruppe

fassen. Das ist behoben und durch einen eigenen Test abgesichert. Die Rückübersetzung ist ein Test, kein Beweis: Fehler, die Generator und Leser gleichermaßen machen, blieben unentdeckt; deshalb vergleichen die Tests zusätzlich Paketurteile (Sätze 10 und 11) mit den zurückgelesenen Regeln. Die Ergebnisse zum Subgraph Algorithmus stehen in Bemerkung 10.

Kapitel 5

Verwendungsbeispiele

5.1 Das Konfigurationsbeispiel im Detail

Die Datei `examples/config.example.yaml` beschreibt ein fiktives Firmennetz. Sie ist Vorlage, Referenz (jeder Schlüssel kommt mindestens einmal vor) und Testgrundlage (`test_example_configuration_is_valid`).

5.2 Die Datei

Listing 5.1: `examples/config.example.yaml`

```yaml
# Beispielkonfiguration für subnetplan
base_network: 10.20.0.0/16      # zugewiesener Bereich (IPv4 oder IPv6)
growth_factor: 1.25             # Wachstumsreserve pro Gruppe (>= 1.0)
allocation_strategy: vlsm       # vlsm = größte Gruppe zuerst | fixed = einheitliche Präfixlänge
gateway_position: first         # first | last | none

groups:
  - name: Server
    hosts: 40
    vlan_id: 100
    priority: 0                 # kleinere Zahl = früher im Adressraum (bei gleicher Größe bzw. bei fixed)
    access: [Drucker]

  - name: Entwicklung
    hosts: 120
    access: [Server, Drucker]

  - name: HR
    hosts: 25
    access: [Drucker]

  - name: Drucker
    hosts: 12
```

```
24
25    - name: Gäste
26      hosts: 60                    # kein access: keine internen
          Ziele
```

5.3 YAML in zwei Minuten

YAML ist ein Textformat für Einstellungen. Zeilen der Form `schluessel: wert` sind Einstellungen. Ein `#` beginnt einen Kommentar, den das Programm ignoriert. Ein Bindestrich `-` am Zeilenanfang beginnt ein neues Listenelement; hier ist jedes Element eine Gruppe. Eingerückte Zeilen gehören zum darüberstehenden Element. Eckige Klammern wie in `[Server, Drucker]` schreiben eine kurze Liste in eine Zeile. Es dürfen nur Leerzeichen zum Einrücken verwendet werden, keine Tabulatoren.

5.4 Die vier Kopfzeilen

Schlüssel	Wert im Beispiel	Bedeutung
`base_network`	`10.20.0.0/16`	Der zugewiesene Adressbereich. Ein `/16` hat $2^{16} = 65\,536$ Adressen von `10.20.0.0` bis `10.20.255.255`. Der Bereich `10.x.x.x` ist für private Netze reserviert [3]. Pflichtfeld ohne Standardwert.
`growth_factor`	`1.25`	Wachstumsreserve: Jede Gruppe darf um 25 % wachsen, ohne dass das Netz umgebaut werden muss. Mindestens 1,0.
`allocation_strategy`	`vlsm`	Jede Gruppe bekommt ein Netz passender Größe, das größte zuerst. Die Alternative `fixed` gibt allen Gruppen die Größe der größten.
`gateway_position`	`first`	Das Gateway liegt auf der ersten freien Adresse (die erste Adresse nach der Netzadresse). Alternativen: `last` oder `none`.

5.5 Die fünf Gruppen

Gruppe	Hosts	`access` (darf zugreifen auf)	Besonderheit
`Server`	40	`Drucker`	feste `vlan_id: 100`, `priority: 0`
`Entwicklung`	120	`Server`, `Drucker`	größte Gruppe, zwei Ziele
`HR`	25	`Drucker`	Standardpriorität 0
`Drucker`	12	niemanden	reiner Dienst
`Gäste`	60	niemanden	kein `access`: kein interner Zugriff

`access` ist eine Erlaubnisliste und gilt nur in eine Richtung: `Entwicklung` darf auf `Server` zugreifen, `Server` aber nicht auf `Entwicklung`. Dass `Drucker` und `Gäste` keine `access`-Zeile haben, heißt: Sie bauen keine Verbindungen in andere Netze auf.

5.6 Schritt 1: Bedarf und Präfixlänge

Das Programm rechnet je Gruppe $r = \lceil 1{,}25\,h \rceil + 3$ (drei Sonderadressen: Netz, Broadcast, Gateway) und sucht die kleinste Zweierpotenz $s \geq r$:

Gruppe	h	$\lceil 1{,}25h \rceil$	r	$k = \lceil \log_2 r \rceil$	$s = 2^k$	Präfix $32 - k$
Entwicklung	120	150	153	8	256	`/24`
Gäste	60	75	78	7	128	`/25`
Server	40	50	53	6	64	`/26`
HR	25	32	35	6	64	`/26`
Drucker	12	15	18	5	32	`/27`

Bei `HR` ist $25 \cdot 1{,}25 = 31{,}25$, aufgerundet 32. Die Summe der Blockgrößen ist $S = 256 + 128 + 64 + 64 + 32 = 544 \leq 65\,536$, der Plan ist also nach Satz 3 möglich. Zum Vergleich: Die Summe der Bedarfe ist $\sum r_i = 153 + 78 + 53 + 35 + 18 = 337$; die Blöcke belegen nach Satz 7 weniger als das Doppelte davon ($544/337 \approx 1{,}61$).

5.7 Schritt 2: Reihenfolge

Gruppe	Präfix p	Priorität π	Position in der Datei
Entwicklung	24	0	2
Gäste	25	0	5
Server	26	0	1
HR	26	0	3
Drucker	27	0	4

Sortiert wird zuerst nach dem Präfix (kleiner heißt größerer Block, also früher), dann nach der Priorität und zuletzt nach der Position in der Datei. Daraus folgt die Reihenfolge `Entwicklung`, `Gäste`, `Server`, `HR`, `Drucker`. Drei Dinge sind bemerkenswert:

- `Gäste` steht in der Datei zuletzt, kommt aber an zweiter Stelle, weil ein `/25` größer ist als ein `/26` oder `/27`.
- `Server` und `HR` brauchen beide ein `/26` und haben die Priorität 0. Dann entscheidet die Dateireihenfolge, `Server` steht vor `HR`. Setzt man bei `Server` `priority: 1`, rückt `HR` nach vorn: `HR` erhält `10.20.1.128/26` und `Server` `10.20.1.192/26` (so vom Programm berechnet).
- Die Reihenfolge der Gruppen in der Datei ist sonst bedeutungslos. Sie wirkt nur als letzter Tie-Breaker und bei der automatischen VLAN-Vergabe.

5.8 Schritt 3: Der Zeiger

Die Adresse 10.20.0.0 entspricht der Zahl $a_B = 10 \cdot 2^{24} + 20 \cdot 2^{16} = 169\,082\,880$. Der Zeiger c wandert, in Adressen ab a_B gerechnet, so:

Gruppe	Start (Offset)	Größe	Start als Adresse
Entwicklung	0	256	`10.20.0.0`
Gäste	256	128	`10.20.1.0`
Server	384	64	`10.20.1.128`
HR	448	64	`10.20.1.192`
Drucker	512	32	`10.20.2.0`
Zeiger am Ende	544	–	`10.20.2.32`

Jeder Start ist durch die Blockgröße teilbar ($384 = 6 \cdot 64$, $448 = 7 \cdot 64$, $512 = 16 \cdot 32$): Das ist die Lückenlosigkeit aus Satz 2 an einem Beispiel, es wird nie aufgerundet.

5.9 Schritt 4: VLAN-IDs

`Server` behält die feste ID 100. Alle anderen erhalten in Dateireihenfolge die kleinsten freien Werte aus $10, 20, 30, \ldots$: `Entwicklung` 10, `HR` 20, `Drucker` 30, `Gäste` 40. Die Zahl 100 wird dabei übersprungen, falls sie später an der Reihe wäre (Lemma 6).

5.10 Das Ergebnis

Der Aufruf `subnetplan examples/config.example.yaml` gibt aus:

Listing 5.2: Ausgabe des Beispiels

```
Gruppe        VLAN  Netz/CIDR        Subnetzmaske      Gateway       Host-Range
                     Broadcast     Nutzbar  Bedarf
------------  ----  --------------  ----------------  ------------
    -------------------------  -----------  -------  ------
Entwicklung   10    10.20.0.0/24    255.255.255.0     10.20.0.1     10.20.0.2 - 10.20.0.254
       10.20.0.255  253      120
Gäste         40    10.20.1.0/25    255.255.255.128   10.20.1.1     10.20.1.2 - 10.20.1.126
       10.20.1.127  125      60
Server        100   10.20.1.128/26  255.255.255.192   10.20.1.129   10.20.1.130 -
    10.20.1.190  10.20.1.191  61       40
HR            20    10.20.1.192/26  255.255.255.192   10.20.1.193   10.20.1.194 -
    10.20.1.254  10.20.1.255  61       25
Drucker       30    10.20.2.0/27    255.255.255.224   10.20.2.1     10.20.2.2 - 10.20.2.30
        10.20.2.31   29       12

Basisnetz: 10.20.0.0/16 (65536 Adressen)
Belegt: 544 (0.83 %), frei: 64992
Freie Bereiche: 10.20.2.32/27, 10.20.2.64/26, 10.20.2.128/25, 10.20.3.0/24, 10.20.4.0/22,
     10.20.8.0/21, 10.20.16.0/20, 10.20.32.0/19, 10.20.64.0/18, 10.20.128.0/17
```

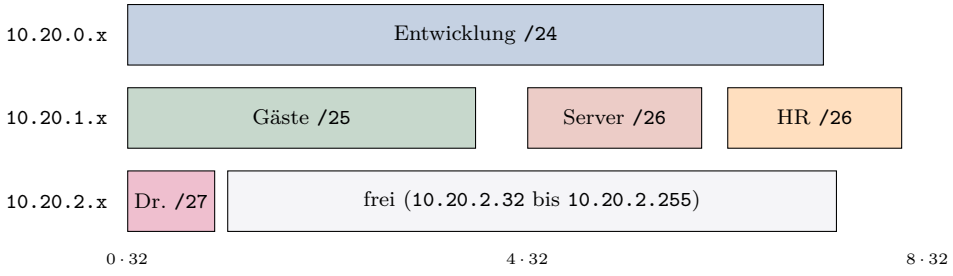


Abbildung 5.1: Belegung der ersten drei `/24`-Zeilen. Jede Zeile hat 256 Adressen; eine Einheit der Breite entspricht 32 Adressen. Die Blöcke schließen lückenlos aneinander an.

5.11 Nutzbar, Bedarf und Reserve

Die Spalte „Nutzbar“ liegt immer über dem Bedarf, weil Blockgrößen Zweierpotenzen sind und weil die Wachstumsreserve eingerechnet ist. Für `Entwicklung` sind $253 = 256 - 3$ Adressen nutzbar (Lemma 2); davon werden zunächst 120 gebraucht, 133 sind Reserve. Entsprechend: `Gäste` $125 - 60 = 65$, `Server` $61 - 40 = 21$, `HR` $61 - 25 = 36$, `Drucker` $29 - 12 = 17$.

5.12 Ein Blick in die Binärdarstellung

Das Netz `10.20.1.128/26` hat im letzten Byte die Bits `10|000000`: Die ersten beiden Bit (`10`) gehören zum Netzteil, die sechs übrigen zur Hausnummer. `HR` (`10.20.1.192/26`) hat im letzten Byte `11|000000`. Beide `/26`-Netze sind also die beiden Hälften des `/25`-Netzes `10.20.1.128/25` (Lemma 1, Abbildung 2.1), und `Gäste` `10.20.1.0/25` ist die andere Hälfte der Zeile `10.20.1.x`.

5.13 Freie Bereiche und Wachstum

Belegt sind 544 von 65 536 Adressen (0,83 %), frei sind 64 992. Die zehn freien Bereiche sind nach Satz 8 die Binärdarstellung von 64 992 (Beispiel in Abschnitt 3.5).

Angenommen, eine neue Gruppe `Labor` mit 100 Hosts kommt hinzu: $r = 125 + 3 = 128$, also ein `/25`. Nach Korollar 1 passt sie, ohne bestehende Netze zu verschieben, denn $128 \leq 64\,992$. Der Beweis zeigt auch wohin: in den kleinsten ausreichend großen freien Block, das ist `10.20.2.128/25`. Plant man dagegen alles *neu* (Gruppe `Labor` an die Datei anhängen und `subnetplan` laufen lassen), so erhält `Labor` das Netz `10.20.1.128/25`, und `Server`, `HR` und `Drucker` wandern nach `10.20.2.0/26`, `10.20.2.64/26` und `10.20.2.128/27`: Das Ergebnis ist gleich gut (gleicher Platzbedarf), aber bestehende Geräte müssten umnummeriert werden. Das Programm plant derzeit immer neu; die inkrementelle Variante ist ein Ausblick.

5.14 Die Firewall-Regeln

Aus den `access`-Einträgen ergeben sich genau vier Zugriffspaare:

$$P = \{(\text{Entwicklung}, \text{Server}),\ (\text{Entwicklung}, \text{Drucker}),\\ (\text{Server}, \text{Drucker}),\ (\text{HR}, \text{Drucker})\}.$$

Der Aufruf `subnetplan examples/config.example.yaml -firewall nftables` fügt hinzu:

Listing 5.3: nftables-Regeln für das Beispiel

```
# Generiert von subnetplan: Standard ist Verwerfen, erlaubt sind nur die Gruppenzugriffe.
table inet subnet_acl
delete table inet subnet_acl
table inet subnet_acl {
    chain forward {
        type filter hook forward priority 0; policy drop;
        ct state established,related accept
        ip saddr 10.20.0.0/24 ip daddr 10.20.1.128/26 accept comment "Entwicklung ->
            Server"
```

```
        ip saddr 10.20.0.0/24 ip daddr 10.20.2.0/27 accept comment "Entwicklung ->
            Drucker"
        ip saddr 10.20.1.128/26 ip daddr 10.20.2.0/27 accept comment "Server -> Drucker"
        ip saddr 10.20.1.192/26 ip daddr 10.20.2.0/27 accept comment "HR -> Drucker"
    }
}
```

Zeile für Zeile:

- `table inet subnet_acl` und `delete table inet subnet_acl`: Die Tabelle wird angelegt und sofort gelöscht, damit die Datei beliebig oft eingespielt werden kann und stets den alten Stand ersetzt.
- `type filter hook forward priority 0; policy drop;`: Die Kette prüft weitergeleitete Pakete (nicht Pakete an die Firewall selbst). Was nicht ausdrücklich erlaubt ist, wird verworfen.
- `ct state established,related accept`: Antworten auf bereits aufgebaute Verbindungen sind erlaubt. Deshalb können `Server` und `Drucker` antworten, obwohl sie selbst nicht zugreifen dürfen.
- Die vier `ip saddr ... accept`-Zeilen sind die Freigaben aus P, jeweils von Quell- zu Zielnetz.

Nach Satz 10 darf `Entwicklung` neue Verbindungen zu `Server` und `Drucker` aufbauen, `Server` und `HR` zu `Drucker`, sonst niemand zu niemandem. `Gäste` und `Drucker` tauchen nie als Quelle auf. Wichtig: Alles andere wird verworfen, *auch* die Weiterleitung ins Internet. Brauchen die `Gäste` Internetzugang, muss er zusätzlich ergänzt werden.

Mit `-firewall iptables` entstehen dieselben Freigaben als Befehle in der Kette `SUBNET_ACL`:

Listing 5.4: iptables-Regeln für das Beispiel

```
iptables -N SUBNET_ACL
iptables -A SUBNET_ACL -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -A SUBNET_ACL -s 10.20.0.0/24 -d 10.20.1.128/26 -m comment --comment "
    Entwicklung -> Server" -j ACCEPT
iptables -A SUBNET_ACL -s 10.20.0.0/24 -d 10.20.2.0/27 -m comment --comment "Entwicklung
    -> Drucker" -j ACCEPT
iptables -A SUBNET_ACL -s 10.20.1.128/26 -d 10.20.2.0/27 -m comment --comment "Server ->
    Drucker" -j ACCEPT
iptables -A SUBNET_ACL -s 10.20.1.192/26 -d 10.20.2.0/27 -m comment --comment "HR ->
    Drucker" -j ACCEPT
iptables -A SUBNET_ACL -j DROP
iptables -I FORWARD 1 -j SUBNET_ACL
```

Die letzte Zeile hängt die Kette an Position 1 in `FORWARD` ein. Mit `-firewall cisco` erhält jede Gruppe eine eigene eingehende ACL am Interface `Vlan<ID>`; `0.0.0.255` ist die *Wildcard-Maske* (die invertierte Subnetzmaske):

Listing 5.5: Cisco-ACLs für das Beispiel

```
ip access-list extended ACL-10-ENTWICKLUNG
 remark Entwicklung -> Server
 permit ip 10.20.0.0 0.0.0.255 10.20.1.128 0.0.0.63
 remark Entwicklung -> Drucker
 permit ip 10.20.0.0 0.0.0.255 10.20.2.0 0.0.0.31
 deny ip any any
interface Vlan10
 ip access-group ACL-10-ENTWICKLUNG in
ip access-list extended ACL-40-GÄSTE
 deny ip any any
interface Vlan40
```

```
 ip access-group ACL-40-GÄSTE in
ip access-list extended ACL-100-SERVER
 remark Server -> Drucker
 permit ip 10.20.1.128 0.0.0.63 10.20.2.0 0.0.0.31
 deny ip any any
interface Vlan100
 ip access-group ACL-100-SERVER in
ip access-list extended ACL-20-HR
 remark HR -> Drucker
 permit ip 10.20.1.192 0.0.0.63 10.20.2.0 0.0.0.31
 deny ip any any
interface Vlan20
 ip access-group ACL-20-HR in
ip access-list extended ACL-30-DRUCKER
 deny ip any any
interface Vlan30
 ip access-group ACL-30-DRUCKER in
```

Die ACL von `Gäste` (`ACL-40-GÄSTE`) und die von `Drucker` enthalten nur `deny ip any any`. Nach Satz 11 sind Cisco-ACLs zustandslos: Die Antwort des `Druckers` an `Entwicklung` tritt am Interface `Vlan30` ein und wird dort verworfen. Damit Verbindungen zustande kommen, muss der Rückverkehr von Hand ergänzt werden (Ausblick).

5.15 Varianten ohne Bearbeiten der Datei

– `-strategy fixed`: Alle Gruppen bekommen die Größe der größten (`/24`), die Reihenfolge folgt nur der Priorität und der Datei: `Server 10.20.0.0/24`, `Entwicklung 10.20.1.0/24`, `HR 10.20.2.0/24`, `Drucker 10.20.3.0/24`, `Gäste 10.20.4.0/24`. Belegt sind $5 \cdot 256 = 1280$ statt 544 Adressen; der Mehrverbrauch ist $W = 1280 - 544 = 736$ (Satz 6).

– `-base-network 192.168.0.0/16`: Dasselbe Beispiel in einem anderen Bereich; `Entwicklung` erhält `192.168.0.0/24`, `Gäste 192.168.1.0/25`, und so weiter.

– `-growth-factor 1.0 -gateway none`: Ohne Reserve und ohne Gateway entstehen kleinere Netze: r ist $122, 62, 42, 27, 14$, also `Entwicklung 10.20.0.0/25`, `Server 10.20.0.128/26`, `Gäste 10.20.0.192/26`, `HR 10.20.1.0/27` und `Drucker 10.20.1.32/28`. Das sind nur $128 + 64 + 64 + 32 + 16 = 304$ Adressen. Hier liegt `Server` vor `Gäste`, weil beide ein `/26` brauchen und `Server` in der Datei vorn steht.

– `-format json -o plan.json` oder `-format csv -o plan.csv`: maschinenlesbare Ausgabe zur Weiterverarbeitung.

5.16 Typische Fehler

Fehler werden gemeldet, ohne dass ein Plan ausgegeben wird (Rückgabewert 1):

```
Fehler: Gruppe 'Server': access verweist auf unbekannte Gruppen ['Drucker']
Fehler: Platzmangel: Gruppe 'Gaeste' (/25) passt nicht mehr in 10.20.0.0/24
base_network '10.0.0.5/24' ist ungültig: 10.0.0.5/24 has host bits set
growth_factor muss eine Zahl >= 1.0 sein, erhalten 0.9
Doppelte VLAN-IDs: [5]
```

Die zweite Meldung entsteht, wenn man das Beispiel auf ein `/24` verkleinert: $256+128 > 256$ (Satz 3).

5.17 Eigene Konfiguration ableiten

1. Datei kopieren, zum Beispiel mit `cp` nach `mein-netz.yaml`.
2. `base_network` auf den zugewiesenen Bereich setzen.
3. Unter `groups` die eigenen Gruppen mit `name` und `hosts` eintragen, bei Bedarf `vlan_id` und `priority`.
4. In `access` festlegen, wer wen erreichen darf. Die Namen müssen genau einer Gruppe entsprechen.
5. `subnetplan mein-netz.yaml` ausführen und das Ergebnis prüfen, bevor Regeln eingespielt werden.

5.18 Weitere Code-Beispiele

5.19 Maschinenlesbare Ausgabe

Listing 5.6: Auszug der CSV-Ausgabe (`-format csv`)

```
group,vlan_id,network,netmask,gateway,first_host,last_host,broadcast,usable_hosts,
    requested_hosts,access
Entwicklung
    ,10,10.20.0.0/24,255.255.255.0,10.20.0.1,10.20.0.2,10.20.0.254,10.20.0.255,253,120,
    Server;Drucker
Gäste,40,10.20.1.0/25,255.255.255.128,10.20.1.1,10.20.1.2,10.20.1.126,10.20.1.127,125,60,
```

Die JSON-Ausgabe (`-format json`) enthält zusätzlich die Kennzahlen des Plans: `"total_addresses": 65536`, `"allocated_addresses": 544`, `"free_addresses": 64992`, `"utilization_percent": 0.83`, die Liste `"free_ranges"` und je Subnetz ein Objekt mit allen Feldern der CSV-Spalten.

Zusammen mit `-firewall` muss bei `-format json` oder `csv` eine Ausgabedatei (`-o` oder `-firewall-output`) angegeben werden, damit die Standardausgabe maschinenlesbar bleibt; andernfalls meldet das Programm einen Fehler.

5.20 Verwendung als Python-Bibliothek

Listing 5.7: Planung aus Python heraus

```
from src.config import build_config, load_config_file
from src.planner import plan_subnets
from src.firewall import render_firewall

cfg = build_config(load_config_file("examples/config.example.yaml
    "))
plan = plan_subnets(cfg)

for s in plan.subnets:
    print(f"{s.group:12} {str(s.network):16} {s.usable_hosts:4}")
print(plan.allocated_addresses, plan.free_addresses, round(plan.
    utilization, 2))
```

```
print(render_firewall(plan, "iptables"))
```

Die Ausgabe beginnt mit den fünf Zeilen `Entwicklung 10.20.0.0/24 253`, `Gäste 10.20.1.0/25 125`, `Server 10.20.1.128/26 61`, `HR 10.20.1.192/26 61`, `Drucker 10.20.2.0/27 29`, danach folgt `544 64992 0.83` und das iptables-Skript. Nach `pip install -e .` lautet der Paketname `subnetplan` statt `src`.

5.21 IPv6

Für `base_network: 2001:db8:abcd::/48` mit `Entwicklung` (120 Hosts) und `Drucker` (12 Hosts) entstehen `2001:db8:abcd::/120` (Gateway `2001:db8:abcd::1`, 254 nutzbar) und `2001:db8:abcd::100/123` (30 nutzbar). Es gibt keine Broadcast-Adresse ($o = 2$ statt 3): $r = 150 + 2 = 152$ liefert $k = 8$, also `/120`. Die freien Bereiche bestehen hier aus vielen Blöcken, bis zu $128 - 48 = 80$ (Satz 8). Das Ergebnis ist mathematisch korrekt, entspricht aber nicht der Betriebspraxis, in der jedes IPv6-Subnetz ein `/64` ist [8]; der Ausblick behandelt dies (Satz 18).

5.22 Wiederholte Zugriffsziele

Steht in `access` ein Ziel doppelt (`access: [B, B]`), erzeugt der Generator die Freigabe doppelt. Das ist harmlos, weil das Urteil nur von der Menge P abhängt (Satz 10), aber unnötig; Satz 15 behandelt die Bereinigung.

Kapitel 6

Experimentelle Ergebnisse

Hardwareplattform der lokal ausgeführten Experimente: Rechner `WBIS-H207085` von LENOVO, Modell `10T8S6LB00`, mit 7,87 GB Arbeitsspeicher; ein Intel-Core-i3-8100T-Prozessor mit 3,10 GHz, vier physischen Kernen und vier logischen Prozessoren (x64); Microsoft Windows 11 Pro, 64-Bit, Version 10.0.26200.

Dieses Kapitel wertet den Lauf des Experimentprogramms `src/experiments.py` vom 19. Oktober 2026 aus. Die Experimente ergänzen die Beweise und die zuvor beschriebenen Tests: Sie prüfen ausgewählte Aussagen an endlich vielen reproduzierbaren Zufallseingaben und messen Größen, die sich nicht allein aus Korrektheitssätzen ergeben, etwa Laufzeit, Auslastung und Länge erzeugter Firewall-Regeln. Ein bestandener Versuch ist daher eine empirische Bestätigung für die verwendeten Eingaben, kein Beweis für alle möglichen Konfigurationen.

6.1 Versuchsaufbau und Reproduzierbarkeit

Der vollständige Lauf verwendete den Startwert 2026 und die im Programm als Standard festgelegten Stichprobengrößen. Jeder Versuch erhält einen eigenen Zufallszahlengenerator, dessen Startwert aus dem globalen Seed und dem Namen des Experiments gebildet wird. Damit bleiben die Eingaben bei Wiederholung mit gleichem Programmstand, Seed und Umfang reproduzierbar; die Laufzeiten sind wegen Betriebssystem- und Prozesszustand nicht deterministisch. Die Zusammenfassung dokumentiert Python 3.13.16, Windows 11 (Plattformkennung `10.0.26300`), Start- und Endzeit sowie den Seed. Insgesamt dauerte der Lauf rund 40,09 Sekunden.

Experiment	Umfang laut Runner	Dauer	Urteil
Laufzeit	geplant bis $n = 5000$, je drei Messungen	0,38 s	Fehler
Auslastung	4000 Pläne	5,18 s	bestanden
Wachstumsfaktor	200000 Arithmetikproben, elf Faktorwerte	2,67 s	bestanden
Packbarkeit	4000 Fälle	0,41 s	bestanden
Erweiterung	1500 Pläne	1,46 s	bestanden
Regelgruppierung	40 Pläne je Kombination aus n und Dichte	23,48 s	bestanden
Rückübersetzung	3600 Prüfungen	1,14 s	bestanden
Hierarchie	1000 Pläne	1,53 s	bestanden
Verdopplung	300 Pläne	0,44 s	bestanden
IPv6	geplant bis $n = 2000$, zusätzlich 100 Pläne	0,02 s	Fehler
Policy	6000 Paare (3000 enthalten, 3000 unabhängig)	3,24 s	bestanden

Neun der elf Einträge wurden als bestanden markiert. Der Gesamtlauf selbst endete dennoch mit `all_passed = false`, da Laufzeit- und IPv6-Experiment eine Ausnahme auslösten. Die Laufzeiten in der Tabelle messen die Dauer des jeweiligen Experiments einschließlich der Erzeugung und Prüfung seiner Testfälle, nicht nur die Laufzeit des Planungsalgorithmus. Insbesondere ist die Gruppierungsmessung mit rund 23,48 Sekunden der teuerste Versuch; ihr Ergebnis ist eine Ausgabegrößenmessung und kein Laufzeitbenchmark des Planers.

6.2 Adressbedarf, Auslastung und Packbarkeit

Auslastung von VLSM und festen Präfixen. Für 4000 zufällig erzeugte Pläne mit logarithmisch verteilten Hostzahlen wurden 27933 einzelne Blockauslastungen $u = r/s$ erfasst. Ihr Mittelwert beträgt 0,7391 (Median 0,7344), gegenüber dem theoretischen Erwartungswert 0,7213. Die Abweichung von 0,0178 entspricht etwa 1,78 Prozentpunkten und liegt unter der im Experiment verwendeten Toleranz von drei Prozentpunkten. Eine unabhängige Monte-Carlo-Referenzrechnung meldet 0,7204 und liegt damit nahe am theoretischen Wert. Die beobachtete minimale Auslastung ist 0,50024; kein Block lag bei oder unter der im Satz zum Verschnitt ausgeschlossenen Grenze 1/2. Zusammen mit null Fehlern des Planverifikators und 4000/4000 Treffern der Formel für den Mehrverbrauch fester Präfixe stützt dies die erwarteten Aussagen.

Die Gegenüberstellung von `vlsm` und `fixed` quantifiziert den Einfluss stark unterschiedlicher Gruppengrößen: Der Quotient aus der bei fester Präfixlänge belegten und der bei VLSM belegten Adresszahl lag im Mittel bei 3,658 und im Median bei 3,459. Im ungünstigsten beobachteten Plan war der feste Plan um den Faktor 10,63 größer; in einzelnen Fällen waren beide Strategien gleich groß. Das ist kein universeller Faktor für beliebige Konfigurationen, sondern die Verteilung der hier erzeugten Stichprobe. In allen 4000 Fällen entsprach der zusätzliche Verbrauch exakt der im Code geprüften Formel `fixed\_extra\_waste`.

Wachstumsfaktor und exakte Arithmetik. Der Faktorsweep betrachtet ein festes Fünf-Gruppen-Beispiel mit Faktoren von 1,0 bis 2,0 in Schritten von 0,1. Die belegte Blocksumme ist nicht stetig: Sie steigt von 304 Adressen bei 1,0 auf 512 bei 1,1, 544 bei 1,2 und 608 bei 1,6. Dazwischen bleibt sie konstant. Ursache sind die diskreten Blockgrößen als Zweierpotenzen: Ein kleiner Mehrbedarf ändert die Zuteilung erst dann, wenn eine Gruppe die Kapazität ihres bisherigen Blocks überschreitet; dann kann der zugeteilte Block sprunghaft auf die nächste Größe anwachsen. Ein größerer Wachstumsfaktor garantiert daher nicht in jedem einzelnen Schritt eine höhere Gesamtauslastung. Im Beispiel mit Faktor 1,25 werden 544 Adressen belegt, also 0,83 % des `/16`-Basisnetzes.

Zusätzlich wurden 200000 Paare aus Hostzahl und Faktor im Bereich 0,1 bis 2,0 mit einer dezimalen Referenz verglichen. Werte unter 1 sind keine gültigen Konfigurationswerte für den Wachstumsfaktor; sie prüfen hier ausschließlich die Arithmetikfunktion. Die direkte Rechnung mit binärer Fließkommaarithmetik und anschließender Aufrundung wich in 48 Fällen um eine Adresse ab. Die untersuchte gerundete Fließkommavariante und die exakte Implementierung wichen in keiner Stichprobe ab. Die Null-Abweichungen sind keine Aussage über alle reellen Zahlen; sie belegen für diesen Eingabebereich aber, warum die Implementierung die Dezimalzahl kontrolliert und nicht unkritisch ein binäres Fließkommaergebnis aufrundet.

Packbarkeitskriterium. In 4000 kleinen Adressräumen mit jeweils 32 Adressen wurde die schnelle Packbarkeitsprüfung mit einer unabhängigen vollständigen Backtracking-Suche verglichen. Beide Verfahren stimmten in allen 4000 Fällen überein. In 2686 Fällen meldete die Prüfung die Aufträge als packbar; anschließend waren sämtliche konstruktiv erzeugten Zuteilungen innerhalb der freien Blöcke und paarweise überlappungsfrei (2686/2686). Das ist ein besonders aussagekräftiger Test für die notwendige und hinreichende Bedingung, weil die Referenzsuche nicht dieselbe schnelle Entscheidungsroutine wiederverwendet. Die Schlussfolgerung bleibt dennoch auf das getestete Universum und die dort generierten Blockgrößen beschränkt.

6.3 Erweiterung und hierarchische Planung

Inkrementelle Erweiterung. Für 1500 vorhandene Pläne wurde jeweils eine neue Gruppe angefordert. Die Vorhersage anhand der gesamten freien Adresszahl stimmte in allen Fällen mit dem tatsächlichen Erfolg oder der Zurückweisung überein. 1226 Erweiterungen gelangen; in allen diesen Fällen blieben sämtliche vorhandenen Subnetze unverändert und der erweiterte Plan bestand die Verifikation. Die übrigen 274 Anforderungen passten nicht. Das Experiment bestätigt somit nicht bloß, dass genügend Platz in den erfolgreichen Fällen vorhanden war, sondern auch, dass die verwendete Kapazitätsbedingung die beobachtete Grenze korrekt vorhersagte.

Standortblöcke und Routenaggregation. In 1000 Plänen mit zwei bis sechs Standorten hielten alle Pläne die geprüften Schranken für Standortblöcke ein und bestanden die Planverifikation. Die Summe der Standortblockgrößen geteilt durch die Summe der Gruppenblockgrößen lag im Mittel bei 1,487 (Median 1,516; Maximum 1,959). Der Standortzuschnitt kostet damit in diesen Zufallsplänen zusätzliche reservierte Adressen, blieb aber stets unter dem im Experiment geprüften Faktor 2.

Das Verhältnis der Zahl aggregierter Standortpräfixe zur Zahl einzelner Gruppenpräfixe lag im Mittel bei 0,402 und im Median bei 0,333. In der Stichprobe waren damit im Mittel ungefähr 60 % weniger Standort-Routen als Gruppen-Routen nötig. Die Spannweite von 0,1 bis 1,0 ist für die Interpretation wichtig: Bei günstiger Adresslage lassen sich viele Gruppen zu wenigen Routen zusammenfassen; im schlechtesten beobachteten Fall entsteht keine Reduktion. Der Messwert beschreibt die erzeugten Pläne, nicht die Zahl tatsächlich benötigter Routen in einem konkreten Netzbetrieb.

Verdopplung an Ort und Stelle. Über 300 Pläne wurden 1336 Gruppen geprüft. Eine Verdopplung war für 213 Gruppen möglich: 68 bei Gateway-Position `first`, 71 bei `last` und 74 bei `none`. Alle erfolgreichen Operationen lieferten gültige Pläne und erhielten die VLAN-ID; alle nicht zulässigen Operationen wurden abgewiesen. Bei `first` und `none` änderte sich bei keiner erfolgreichen Verdopplung die Hostadressierung. Bei `last` wurde erwartungsgemäß in allen 71 Fällen genau eine Hostadresse umnummeriert, weil sich durch die Vergrößerung die Lage der letzten Gateway-Adresse ändert. Das Ergebnis macht die Betriebsbedingung sichtbar: `last` kann eine minimale Umnummerierung erfordern, während die beiden anderen geprüften Gateway-Positionen sie vermeiden.

6.4 Firewall-Regeln und Richtlinien

Gruppierung und Ausgabegröße. Der Gruppierungsversuch erzeugte 600 Pläne: $n \in \{10, 30, 100\}$, fünf Zugriffsdichten und jeweils 40 Wiederholungen. Für alle Konfigurationen entsprach die durch gruppierte Regeln dargestellte Paarmenge exakt der ursprünglichen Zugriffsmatrix. Die Anzahl der Regelgruppen blieb in jeder Messzelle höchstens n. Der Anteil der Regelgruppen an den Zugriffspaaren sinkt bei $n = 100$ von rund 20,1 % bei Dichte 0,05 auf rund 1,12 % bei Dichte 0,9: Je dichter die Matrix, desto mehr einzelne Paare kann eine gemeinsame Zielmenge abdecken.

Die Zeilenzahl hängt jedoch stark vom Ausgabeformat ab. Bei $n = 100$ und Dichte 0,9 sank die mittlere nftables-Ausgabe von 8921 auf 109 Zeilen (etwa 98,8 % weniger). Cisco sank von 18227 auf 9614 Zeilen (etwa 47,3 % weniger). Die gruppierte iptables-Ausgabe stieg dagegen von 8921 auf 9521 Zeilen (etwa 6,7 % mehr). Bei geringer Dichte ($n = 100$, 0,05) stieg sie sogar von 504 auf 1101 Zeilen. Die Option ist somit semantisch korrekt, aber nicht für jeden Renderer eine Komprimierung: nftables kann Mengen kompakt ausdrücken, während andere Darstellungen gruppierte Quell- und Zielmengen in zusätzliche Einzelzeilen expandieren können. Vor einer produktiven Aktivierung sollte daher die konkrete Ausgabedatei des Ziel-Backends betrachtet werden.

Rückübersetzung. Der Rückübersetzungstest prüfte je 300 zufällig erzeugte Pläne für IPv4 und IPv6, drei Firewall-Formate (iptables, nftables und Cisco) und zwei Optionssätze (Standard sowie alle getesteten Optionen). Insgesamt wurden 3600 Konstellationen geprüft; in jeder einzelnen ergab das Zurücklesen der Regeln wieder exakt die ursprüngliche Zugriffspaarmenge (3600/3600). Damit sind Generator und Parser über diese Plattformen und Optionen hinweg konsistent. Es bleibt ein Konsistenztest: Ein Fehler, den Erzeuger und Rückübersetzer gemeinsam machen, kann dadurch allein unentdeckt bleiben; die unabhängigen Semantikprüfungen und die zuvor angegebenen Tests sind deshalb weiterhin erforderlich.

Vergleich mit dem Subgraph Algorithmus. Für 3000 gezielt konstruierte Paare mit $A \subseteq B$ erkannte die Signaturheuristik 1887 als enthalten und verfehlte 1113 (37,1 %). Eine zusätzliche Stichprobe von 3000 unabhängigen Paaren ergab 247 falsche positive Enthalten-Urteile (8,23 %). Die exakte komponentenweise Prüfung erkannte dagegen alle 3000 konstruierten Teilmengenbeziehungen. Das Ergebnis belegt für diese Stichproben sowohl falsch negative als auch falsch positive Heuristikurteile. Es bestätigt damit die methodische Entscheidung, die Heuristik nur als Hinweis zu verwenden und Richtlinienentscheidungen an der exakten Prüfung auszurichten. Das Feld `passed` ist hier kein Qualitätsurteil über die Heuristik: Der Versuch ist bestanden, weil die exakte Prüfung alle konstruierten positiven Fälle erkannte; die gemessenen Fehlurteile der Heuristik sind gerade der erwartete Gegenstand des Experiments.

6.5 Nicht abgeschlossene Experimente und Aussagegrenzen

Das Laufzeitexperiment plante Eingaben mit bis zu 5000 Gruppen und sollte daraus Steigungen im Log-Log-Diagramm bestimmen. Es brach beim ersten nicht mehr darstellbaren Umfang, $n = 1000$, beim automatischen Vergeben der VLAN-IDs mit `PlanError: Keine`

`freie VLAN-ID mehr verfügbar` ab. Die Implementierung vergibt automatische IDs in Zehnerschritten ab 10; damit stehen auf diesem Schema nur die 409 IDs von 10 bis 4090 zur Verfügung. Die bis dahin verarbeiteten Größen $n = 10, 30, 100, 300$ genügen nicht zur geplanten vollständigen Auswertung; die Datei `runtime.json` enthält wegen der Ausnahme gar keine Messpunkte. Aus diesem Lauf lässt sich weder eine empirische Bestätigung der Laufzeitanalyse noch ein Gegenbeispiel zur theoretischen Schranke ableiten. Dafür müsste der Versuch mit passenden expliziten VLAN-IDs, einer erweiterten Vergabestrategie oder einer Gruppenanzahl unterhalb der automatischen Grenze erneut ausgeführt werden.

Auch das IPv6-Experiment scheiterte bei der ersten Größenstufe oberhalb der VLAN-Grenze, $n = 500$, mit derselben Ausnahme. Die Ursache liegt somit in der begrenzten automatischen VLAN-Vergabe und nicht in einem gemeldeten IPv6-Adressierungs- oder Verifikationsfehler. Da der Abbruch vor dem Abschluss der Messreihe erfolgte, enthält `ipv6.json` ebenfalls keine gültigen Skalierungsdaten. Die erfolgreichen IPv6-Rückübersetzungsfälle (1800 der insgesamt 3600 Rückübersetzungen entfielen auf IPv6) liefern weiterhin Evidenz für die Regelsemantik, ersetzen aber keinen eigenständigen IPv6-Skalierungstest.

Als Plausibilitätsanker liefert der zusätzlich ausgeführte Beispielaufruf für `10.20.0.0/16` fünf ausgerichtete, nicht überlappende Teilnetze mit zusammen 544 belegten Adressen. Das sind 0,83 % des Basisnetzes; 64992 Adressen bleiben in zehn maximal zusammengefassten freien Bereichen verfügbar. Die ausgegebenen Hostzahlen decken jeweils den Bedarf der Gruppen ab. Dieser konkrete Lauf illustriert das erwartete Verhalten im Anwendungsfall, ist aber ebenfalls nur ein einzelnes Beispiel.

Insgesamt stützen die bestandenen Prüfungen die getesteten Aussagen zu Blockauslastung, exakter Aufrundung, Packbarkeit, Erweiterung, Hierarchie, Verdopplung und Firewall-Semantik. Der Lauf ist gleichwohl nicht vollständig erfolgreich: Zwei geplante Experimente wurden durch eine technische Kapazitätsgrenze des Testaufbaus abgebrochen. Die angemessene Schlussfolgerung lautet daher nicht „alle Experimente bestanden“, sondern: neun Teilversuche erfüllten ihre jeweiligen Kriterien; Laufzeit- und IPv6-Skalierung müssen nach Anpassung der VLAN-Testdaten erneut gemessen werden. Die vollständigen Einzelwerte und Metadaten stehen in den JSON-Dateien unter `src/results/`.

Kapitel 7

Zusammenfassung

Der Subnetting Algorithmus `sbntgg` bietet eine effiziente und beweisbar korrekte Methode zur Planung von IP-Netzen und zur Erzeugung von Firewall-Regeln. Durch die Verwendung von:

- Blöcken als Zweierpotenzen mit Ausrichtung (Lemma 1, Satz 1)
- der Sortierung nach absteigender Größe, die lückenlos und optimal ist (Sätze 2, 5)
- exakter Beschreibung der freien Bereiche als Binärdarstellung (Satz 8) und verlustfreier Erweiterbarkeit (Korollar 1)
- Default-Deny-Regeln mit bewiesener Semantik (Satz 10)
- Bestandsplanung mit exakter Packbarkeitsbedingung (Sätze 9, 17), Mindestgrößen und eigenem Overhead (Satz 18), zweistufiger Planung (Satz 19) und Verdopplung an Ort und Stelle (Satz 21)
- Regelgruppierung (Satz 16) und einer Rückübersetzung der erzeugten Regeln als Test

erreicht der Algorithmus im Kern die optimale Laufzeit $\Theta(n \log n)$ im Vergleichsmodell und vermeidet naive Ansätze, die $n!$ Reihenfolgen durchsuchen müssten.

Die Experimente in Abschnitt 6 stützen diese Ergebnisse für die untersuchten Stichproben: Neun von elf Teilversuchen bestanden ihre Kriterien. Die mittlere Blockauslastung lag bei 73,91 % gegenüber dem theoretischen Erwartungswert von 72,13 %; die Packbarkeitsentscheidung stimmte in 4000 Fällen mit einer vollständigen Suche überein, und alle 3600 Firewall-Rückübersetzungen waren erfolgreich. Die Richtlinienmessung zeigt zugleich, dass die Subgraph-Signaturheuristik sowohl Teilmengenbeziehungen verfehlen als auch falsche positive Urteile liefern kann; maßgeblich bleibt deshalb die exakte Prüfung.

Eine empirische Laufzeitbestätigung und eine IPv6-Skalierungsaussage lassen sich aus diesem Lauf nicht ableiten: Die beiden entsprechenden Experimente brachen bei $n = 1000$ beziehungsweise $n = 500$ ab, weil die automatische VLAN-Vergabe erschöpft war. Das ist eine Grenze des Versuchsaufbaus, kein nachgewiesener Fehler des Planungsalgorithmus; für diese Messungen ist ein angepasster erneuter Lauf erforderlich.

7.1 Anwendungen

Der Algorithmus eignet sich besonders für:

- Planung von Firmen-, Schul- und Behördennetzen mit VLANs
- Dokumentation und Nachvollziehbarkeit von Adressplänen (CSV/JSON für IPAM-Systeme)
- Erzeugung konsistenter Firewall-Regeln aus einer einzigen Quelle
- Unterricht im Fach Informatik: Subnetting durch Ausprobieren mit Konfigurationsdateien
- Automatisierte Tests von Netzwerk-Entwürfen (Plan als Datei im Versionsverwaltungssystem)

7.2 Erweiterungen: Umsetzung und Ausblick

Dieser Abschnitt enthielt in der ersten Fassung nur Vorschläge. Die meisten sind in Version 1.1.0 umgesetzt; die Beweise bleiben hier stehen und werden durch Angaben zu Modul, Kommandozeile und Prüfung ergänzt (Tabelle 7.2). Offen bleiben Protokolle und Ports, weitere Plattformen, die Integrationen (DHCP, DNS, IPAM), mehrere Netze je Gruppe und die theoretischen Fragen am Ende des Abschnitts.

Erweiterung	Aussage	Modul	Stand
Bereinigung `access`, Warnungen	Satz 15	`access.py`, `checks.py`	umgesetzt
Validierung $O(n)$, Cisco $O(n+a)$	Satz 13	`config.py`, `firewall.py`	umgesetzt
Cisco-Rückverkehr	Satz 11	`firewall.py`	umgesetzt
Exakte Arithmetik	Lemma 5	`exact.py`	umgesetzt
Idempotenz, ASCII, ICMPv6, Logging	—	`firewall.py`	umgesetzt
Regelgruppierung	Satz 16	`access.py`, `firewall.py`	umgesetzt
Bestand, Pools, feste Netze	Satz 17	`blocks.py`, `planner.py`	umgesetzt
Mindestgrößen, Overhead	Satz 18	`planner.py`	umgesetzt
Hierarchie	Satz 19	`hierarchy.py`	umgesetzt
Auslastung, Poisson	Satz 20	`stats.py`	umgesetzt
Erweitern, Verdoppeln	Korollar 1, Satz 21	`extend.py`	umgesetzt
Rückübersetzung	—	`roundtrip.py`	umgesetzt
Richtlinienvergleich	Lemma 10, Bemerkung 10	`policy.py`	umgesetzt
Prüfung eines Plans	Sätze des Kerns	`verify.py`	umgesetzt
Ports, Plattformen, Integrationen	—	—	offen

7.3 Verbesserungen am bestehenden Programm

Die Analyse der ersten Fassung hatte acht Verbesserungsmöglichkeiten sichtbar gemacht; sie sind alle umgesetzt (Umsetzung im Anschluss an die Liste):

1. **Wiederholungen und Selbstbezug in `access` bereinigen.** Der Code erzeugt für wiederholte Ziele identische Regeln, und eine Gruppe darf sich selbst nennen (die Regel wird übersprungen). Eine Warnung hilft dem Anwender, Tippfehler zu erkennen.

2. **Validierung in $O(n)$.** `_check_groups` benötigt wegen `list.count` $O(n^2)$; ein `collections.Counter` oder eine Menge senkt dies auf $O(n)$ (Satz 13).

3. **Cisco-Ausgabe in** $O(n + a)$, indem die Zugriffspaare einmal nach Quelle gruppiert werden, statt den Generator für jedes Subnetz erneut zu durchlaufen.

4. **Rückverkehr für Cisco.** Nach Satz 11 fehlt der Rückverkehr. Möglich sind `permit tcp` N_j N_i `established` in der ACL der Zielgruppe (erlaubt TCP-Pakete mit gesetztem ACK- oder RST-Bit; schützt bekanntermaßen nicht gegen gefälschte ACK-Pakete und deckt UDP und ICMP nicht ab) oder *Reflexive ACLs* beziehungsweise zustandsbehaftete Firewalls auf der Plattform.

5. **Exakte Arithmetik.** Lemma 4 garantiert Korrektheit nur für γ mit höchstens acht Nachkommastellen und $h\gamma \leq 10^6$. Mit `fractions.Fraction` oder `decimal.Decimal` wäre $\lceil \gamma h \rceil$ für alle Eingaben exakt.

6. **Idempotente Skripte.** Das `iptables`-Skript scheitert beim zweiten Einspielen an `-N SUBNET_ACL` (Kette existiert) und fügt die Sprungregel erneut ein. Ein vorheriges Löschen (`-D FORWARD -j SUBNET_ACL`, `-F`, `-X`) analog zum `delete table` bei `nftables` beseitigt das.

7. **Plattform-konforme Namen.** Gruppen wie `Gäste` ergeben ACL-Namen mit Umlaut (`ACL-40-GÄSTE`). Manche Plattformen akzeptieren nur ASCII; eine Transliteration (`Ä` → `AE`) mit Eindeutigkeitsprüfung wäre sinnvoll (Eindeutigkeit bleibt durch die VLAN-ID im Namen gesichert).

8. **ICMPv6 und Pfad-MTU.** Bei IPv6 sollten mindestens die weitergeleiteten Fehlermeldungen (Paket zu groß) durch `related` abgedeckt sein; weitere ICMPv6-Typen können konfigurierbar freigegeben werden.

Umsetzung. (1) `clean_access` in `access.py` entfernt Wiederholungen und Selbstbezüge schon beim Laden und liefert je Fund eine Warnung (`-warnings`). (2) `_check_groups` zählt mit `collections.Counter` in $O(n + a)$. (3) `_pairs_by_source` gruppiert die Zugriffspaare einmal nach Quelle. (4) `-cisco-return established` erzeugt `permit tcp` N_j N_i `established` in der ACL von g_j für jedes Paar $(i, j) \in P$ ohne Gegenpaar $(j, i) \in P$ (`asymmetric_pairs`); die Menge der neuen Verbindungen bleibt P, und UDP sowie ICMP bleiben wie oben beschrieben ungedeckt, worauf `-warnings` hinweist. (5) `exact.py`, Lemma 5. (6) Das `iptables`-Skript entfernt zuerst alle Sprungregeln und die Kette (`-no-idempotent` schaltet das ab). (7) `-ascii-names` transliteriert Namen in ACLs und Kommentaren (`Ä` → `AE`, `ß` → `ss`); die VLAN-ID im Namen sichert die Eindeutigkeit. (8) `-icmpv6 TYPEN` gibt weitere ICMPv6-Typen frei, nur bei IPv6; die Typnamen werden gegen `[a-z0-9-]` geprüft, damit nichts in das Skript eingeschleust werden kann.

Satz 15 (Bereinigung von `access`)**.** Werden Wiederholungen und Selbstbezüge aus den `access`-Listen entfernt, so ändert sich nach Satz 10 das Urteil keines Pakets, und die Zahl der Freigaben sinkt auf genau $|P|$.

Beweis. Das Urteil hängt nach Satz 10 nur von der Menge P ab. Die Bereinigung ändert P nicht (Selbstbezüge sind per Definition nicht in P). Der Generator erzeugt je Element der bereinigten Listen genau eine Regel, das sind $|P|$ viele. □

7.4 Weniger Regeln durch Gruppierung

Gruppen mit gleicher Zielmenge lassen sich zu einer Regel mit Mengen zusammenfassen (`nftables`: `ip saddr { A, B } ip daddr { C }`). Sei $T(i) = \{j \neq i : g_j \in \mathrm{acc}(g_i)\}$ und $\mathcal{T}$ die Menge der verschiedenen nichtleeren $T(i)$.

Satz 16 (Regelgruppierung)**.** Für jedes $T \in \mathcal{T}$ sei $S_T = \{i : T(i) = T\}$ und die Regel R_T: $x \in \bigcup_{i \in S_T} N_i$ und $y \in \bigcup_{j \in T} N_j$. Dann ist das Urteil der Liste $(R_T)_{T \in \mathcal{T}}$ für NEU dasselbe wie in Satz 10, und die Zahl der Regeln ist $|\mathcal{T}| \leq n$.

Beweis. Die Regel R_T trifft zu, wenn es $i \in S_T$ und $j \in T$ gibt mit $x \in N_i$ und $y \in N_j$. Dann ist $j \in T(i)$, also $(i, j) \in P$. Umgekehrt: Ist $(i, j) \in P$, so ist $T(i) \neq \emptyset$, $T := T(i) \in \mathcal{T}$, $i \in S_T$ und $j \in T$, also trifft R_T für Pakete aus N_i nach N_j zu. Somit stimmen die Mengen der angenommenen Pakete überein. Die Anzahl ist $|\mathcal{T}| \leq n$, da jedes $T(i)$ höchstens ein Element von $\mathcal{T}$ ergibt. □

Im Beispiel sind $T(\text{Server}) = T(\text{HR}) = \{\text{Drucker}\}$ und $T(\text{Entwicklung}) = \{\text{Server}, \text{Drucker}\}$: Statt vier Regeln genügen zwei. Eine *minimale* Regelzahl zu finden ist allgemein schwerer: Das Überdecken der Paare P durch möglichst wenige Rechtecke $S \times T$ ist das Problem der Überdeckung durch vollständige bipartite Teilgraphen, das als NP-schwer bekannt ist [13, 12]. Die Gruppierung nach gleichen Zielmengen ist daher eine einfache, polynomielle Heuristik.

Umsetzung. `-group-rules` fasst die Regeln nach Satz 16 zusammen (`rule_groups` in `access.py`). Für das Konfigurationsbeispiel erzeugt `nftables` zwei statt vier Regeln:

```
ip saddr { 10.20.0.0/24 } ip daddr { 10.20.1.128/26, 10.20.2.0/27 } accept comment "
    Entwicklung -> Server,Drucker"
ip saddr { 10.20.1.128/26, 10.20.1.192/26 } ip daddr { 10.20.2.0/27 } accept comment "
    Server,HR -> Drucker"
```

Bei `iptables` entsteht je Regelgruppe ein Paar von `ipset`-Mengen (`hash:net`) und eine Regel mit `-m set -match-set`. Bei Cisco werden nur die Zielmengen zu `object-group network` zusammengefasst, denn die Quelle ist stets das eigene Netz der ACL; bei IPv6 bleibt die Cisco-Ausgabe ungruppiert. Die Rückübersetzung bestätigt für alle drei Plattformen, dass die gruppierten Regeln dieselbe Paarmenge P ergeben.

7.5 Erweiterungen der Planung

Bestandsnetze, feste Blöcke und mehrere Pools. Oft sind Teile des Adressraums bereits vergeben, oder es stehen mehrere Basisnetze zur Verfügung. Sei $\mathcal{M}$ die Menge der maximalen freien Blöcke (aus jeder maximalen freien Lücke die minimale CIDR-Zerlegung, ergänzt um die Pools).

Satz 17 (Planung mit Bestand und mehreren Pools)**.** Die Gruppen mit Blockgrößen $s_1, \ldots, s_n$ lassen sich genau dann überlappungsfrei in den freien Adressraum legen, wenn für alle $j \geq 0$ gilt: $\sum_{i:\, s_i \geq 2^j} s_i \leq \sum_{C \in \mathcal{M},\, |C| \geq 2^j} |C|$. Die Menge $\mathcal{M}$ lässt sich in $O((n+m)\log(n+m) + m\,w)$ Zeit bestimmen (m Bestandsnetze).

Beweis. Die Blöcke der minimalen Zerlegung einer Lücke sind maximal: Läge ein Block C in einem größeren ausgerichteten Block M innerhalb der Lücke, so wäre M nach Lemma 1 durch mindestens zwei Blöcke der Zerlegung überdeckt (denn M ist größer als C und selbst vollständig in der Lücke), und ersetzte man diese durch M, wäre die Zerlegung nicht minimal. Blöcke verschiedener Lücken sind disjunkt, da die Lücken durch belegte Adressen getrennt sind, und ein ausgerichteter Block in der Vereinigung liegt in einer einzigen Lücke. Also ist $\mathcal{M}$ eine Familie disjunkter maximaler Blöcke im Sinne von Satz 9, und dessen Bedingung ist die Behauptung. Die Laufzeit: Sortieren der Bestandsnetze $O(m \log m)$, je Lücke $O(w)$ Blöcke. □

Damit sind exakte Planbarkeitstests mit festen Blöcken (`network: 10.20.5.0/24` in der Gruppe) und mit mehreren Pools möglich. Das konstruktive Verfahren aus dem Beweis von Satz 9 (größte Gruppe zuerst in den kleinsten ausreichend großen freien Block) liefert die Zuteilung.

Bemerkung 8 (Platzierungsreihenfolge)**.** Für das Gelingen genügt es, einen Auftrag in irgendeinen freien Block zu legen, dessen Restkapazität mindestens die Auftragsgröße hat. Bei absteigender Bearbeitung ist der Füllstand eines Blocks eine Summe von Zweierpotenzen, die nicht kleiner als die aktuelle Größe sind, also ein Vielfaches der aktuellen Größe, und die Startadresse (Blockanfang plus Füllstand) ist daher ausgerichtet. Die Zahl der noch freien Plätze der Größe 2^j ist (R_j − bisher Platziertes)$/2^j$ und nach Bedingung ($*$) mindestens so groß wie die Zahl der noch offenen Aufträge dieser Klasse. Daher findet jeder Auftrag einen Block, unabhängig davon, welchen geeigneten Block man früher gewählt hat. `pack` in `blocks.py` wählt den Block mit der kleinsten ausreichenden Restkapazität (bei Gleichstand die kleinere Adresse); das legt nur die Zuteilung fest, nicht ihre Existenz.

Umsetzung. `maximal_free_blocks` bestimmt $\mathcal{M}$ (und meldet überlappende oder außerhalb liegende Bestandsnetze als Fehler), `packable` prüft ($*$), `pack` teilt zu. In der Konfiguration stehen `existing` (Bestand), `pools` (weitere Basisnetze) und je Gruppe `network` (fester Block). Der Vergleich von ($*$) mit einer vollständigen Suche ergab in 4000 von 4000 Fällen dasselbe Ergebnis.

Mindestgrößen, Overhead und Sonderfälle.

Satz 18 (Robustheit gegenüber Mindestgrößen)**.** Ersetzt man die Blockgrößen s_i durch beliebige Zweierpotenzen $s'_i \geq s_i$ (zum Beispiel $s'_i = \max(s_i, 2^{w-p_{\max}})$ für eine Mindestgröße, oder einen anderen Overhead $o_i \geq 0$ je Gruppe), so gelten die Sätze 2, 3, 4 (K1) bis (K5), 5, 8 und 9 unverändert mit s'_i anstelle von s_i. Nicht mehr gültig sind die Minimalität (K6) und die Verschnittschranke von Satz 7.

Beweis. Die Beweise dieser Sätze benutzen von s_i nur, dass es eine Zweierpotenz ist und dass der Block mindestens den Bedarf fasst (für (K4)); beides gilt für $s'_i \geq s_i$. Minimalität und Verschnitt benutzen dagegen $s_i < 2r_i$. □

Das deckt mehrere praktische Wünsche ab: (i) *IPv6-Praxis*: Mit $p_{\max} = 64$ erhält jede Gruppe mindestens ein `/64` [8]; in einem `/48` stehen $2^{16} = 65\,536$ solche Netze bereit. (ii) *Punkt-zu-Punkt-Links*: Mit $o = 0$ ergibt sich für zwei Geräte ein `/31` (IPv4, [6]) beziehungsweise `/127` (IPv6, [7]). (iii) *Cloud-Plattformen*, die je Subnetz eine feste Zahl von Adressen reservieren (bei einigen Anbietern fünf), werden durch ein entsprechendes o abgebildet.

Bemerkung 9 (Adressen bei eigenem Overhead)**.** Bei einem in der Gruppe gesetzten Overhead o reserviert der Code o Adressen in dieser Reihenfolge: die Netzadresse (falls $o \geq 1$), die Broadcast-Adresse (nur IPv4, falls $o \geq 2$), den Gateway (falls die Position nicht `none` ist und noch Reserve bleibt), danach weitere Adressen am Blockanfang. Dann gilt $u = s - o$ (Lemma 2) für jedes $o \leq s - 1$; wegen $s \geq r = \lceil \gamma h \rceil + o$ ist das immer erfüllt. Bei $o = 0$ sind beide Blockenden nutzbar (`/31`, `/127`) und es gibt weder Gateway noch Broadcast. Ist o kleiner als der Standard, fehlt der Gateway oder der Broadcast entsprechend.

Umsetzung. `max_prefix` (`-max-prefix`) legt die Mindestgröße $2^{w-p_{\max}}$ fest, `overhead` den Overhead je Gruppe; beides geht in `group_prefix` und `required_addresses` ein. `verify_plan` prüft deshalb bei solchen Plänen nur die Eigenschaften, die Satz 18 erhält, und nicht (K6) und die Verschnittschranke.

Hierarchische Planung. In größeren Organisationen erhält jeder Standort σ zuerst einen eigenen Block, aus dem seine Gruppen geplant werden. Das ermöglicht Routen-Aggregation: ein einziger Eintrag je Standort statt einer je Gruppe.

Satz 19 (Zweistufige Planung)**.** Sei $D_\sigma = \sum_{i \in \sigma} s_i$ der Bedarf des Standorts σ und $E_\sigma = 2^{\lceil \log_2 D_\sigma \rceil}$ sein Block. Dann gelingt die Planung der Gruppen von σ in einem Block der Größe E_σ nach Satz 3, es gilt $E_\sigma < 2D_\sigma$, und die Gesamtbelegung aller Standorte erfüllt $\sum_\sigma E_\sigma < 2 \sum_i s_i < 4 \sum_i r_i$.

Beweis. Es ist $D_\sigma \leq E_\sigma$, also passt die Summe in den Block, und Satz 3 garantiert die Planbarkeit. Nach Satz 1, angewandt auf D_σ, gilt $E_\sigma < 2D_\sigma$. Summation über die Standorte und Satz 7 liefern die Ketten. □

Umsetzung. Gruppen erhalten den Schlüssel `site` (für alle oder für keine). `plan_hierarchical` legt zuerst die Standortblöcke an (absteigend nach Größe, bei Gleichstand nach kleinster Priorität und Reihenfolge) und plant dann die Gruppen jedes Standorts im eigenen Block mit demselben Algorithmus; bei `fixed` ist $D_\sigma = n_\sigma \max s_i$. Im Beispiel mit den Gruppen von 100, 20 Hosts (Berlin) und 50, 10 Hosts (Hamburg) in `10.0.0.0/16` ist $D_{\text{Berlin}} = 128 + 32 = 160$ und $D_{\text{Hamburg}} = 128 + 16 = 144$; beide Standorte erhalten ein `/24`, und statt vier Routen genügen zwei. Die Kombination mit `existing`, `pools` oder festen Netzen wird abgelehnt.

Dimensionierung der Wachstumsreserve. Der Faktor γ ist bisher eine Konstante. Unter einem Wachstumsmodell (zum Beispiel $H_i \sim \text{Poisson}(\lambda_i)$) kann man pro Gruppe das kleinste k wählen, für das $\Pr[H_i + o \leq 2^k] \geq 1 - \alpha$ gilt. Wie viel Platz die Zweierpotenzen im Mittel verschwenden, zeigt der folgende Satz.

Satz 20 (Mittlere Auslastung)**.** Ist $\log_2 r$ modulo 1 gleichverteilt (eine übliche Annahme für Größen, die über mehrere Größenordnungen streuen), so ist der Erwartungswert der Blockauslastung r/s gleich $\dfrac{1}{2\ln 2} \approx 0{,}721$.

Beweis. Sei $x = \log_2 r$ und $t = \lceil x \rceil - x \in [0, 1)$. Dann ist $r/s = 2^{x-\lceil x \rceil} = 2^{-t}$, und t ist gleichverteilt auf $[0, 1)$ (für nichtganzes x ist $t = 1 - \{x\}$). Also

$$\mathbb{E}[r/s] = \int_0^1 2^{-t}\, dt = \left[-\frac{2^{-t}}{\ln 2}\right]_0^1 = \frac{1 - \frac{1}{2}}{\ln 2} = \frac{1}{2\ln 2}. \qquad \square$$

Im Mittel nutzt ein Block also etwa 72 % seiner Adressen für Bedarf und Reserve; im ungünstigsten Fall liegt die Auslastung knapp über 50 % (Satz 7).

Umsetzung. `plan_statistics` in `stats.py` liefert Blockauslastung je Gruppe, Mittel, Minimum und Verschnitt eines Plans (`-stats`); `simulate_mean_utilization` schätzt den Erwartungswert aus Satz 20 (Ergebnis 0,7210 bei 400 000 Stichproben, Soll 0,7213). `poisson_block_exponent` bestimmt für das Wachstumsmodell das kleinste k mit $\Pr[H + o \le 2^k] \ge 1 - \alpha$; für $\lambda = 100$, $o = 3$ und $\alpha = 0{,}01$ ergibt sich $k = 7$, also ein `/25`.

Inkrementelle Planung und Verdopplung. Nach Korollar 1 verliert eine inkrementelle Ergänzung keine Kapazität gegenüber einer Neuplanung. `subnetplan neue.yaml -extend plan.json` liest einen mit `-format json` erzeugten Plan ein (`plan_from_json`) und legt die neuen Gruppen mit dem Verfahren aus Satz 9 in die freien Blöcke. Alle bestehenden Subnetze, VLAN-IDs und Zugriffslisten bleiben unverändert, neue VLAN-IDs vermeiden die vorhandenen, und `access` der neuen Gruppen darf auf bestehende Gruppen verweisen.

Satz 21 (Verdopplung an Ort und Stelle)**.** Sei N das Netz einer Gruppe, $\hat{N}$ sein Elternblock (doppelte Größe, in einem Pool) und R die andere Hälfte von $\hat{N}$. Ist N die linke Hälfte von $\hat{N}$ und liegt R vollständig im freien Adressraum des Plans, so ist der Plan, in dem N durch $\hat{N}$ ersetzt wird, wieder korrekt im Sinne von Satz 4 (K1) bis (K4); Netzadresse und VLAN-ID bleiben gleich, und bei Gateway `first` oder `none` behalten alle bisherigen Host-Adressen ihre Bedeutung. Ist N die rechte Hälfte oder ist R nicht frei, so ist eine Verdopplung ohne Verschieben anderer Netze oder der Netzadresse nicht möglich.

Beweis. Nach Lemma 1 ist $\hat{N}$ ein ausgerichteter Block, der N und R enthält. Da R frei ist, überlappt $\hat{N}$ kein anderes Subnetz (K3); er liegt im Pool (K2) und ist ausgerichtet (K1). Er ist größer als N, fasst also mindestens den Bedarf (K4). Die Netzadresse von $\hat{N}$ ist die von N, weil N die linke Hälfte ist; Gateway `first` liegt bei Netzadresse +1, und die alte Broadcast-Adresse wird zu einer gewöhnlichen Host-Adresse. Ist N die rechte Hälfte, beginnt $\hat{N}$ links von N und die Netzadresse ändert sich; ist R belegt, entstünde eine Überlappung. $\square$

`can_double` prüft die Bedingung, `double_in_place` führt die Verdopplung aus, `renumbered_hosts` zählt die dabei geänderten Adressen (bei Gateway `last` wandert der Gateway ans neue Blockende). In der Prüfung ergaben 234 von 234 möglichen Verdopplungen einen korrekten Plan ohne Umnummerierung der Hosts (bei Gateway `first` und `none`). Offen bleibt, wie man bei mehreren wachsenden Gruppen die Gesamtzahl der umzunummerierenden Geräte minimiert.

Mehrere Netze je Gruppe. Erlaubt man einer Gruppe mehrere Subnetze (zum Beispiel für sehr große Gruppen oder per Sekundäradresse), so lässt sich ihr Bedarf in Zweierpotenzen zerlegen; jedes Teilnetz hat jedoch eigene Overhead-Adressen. Die Abwägung zwischen weniger Verschnitt und mehr Overhead ist eine offene Optimierungsfrage.

7.6 Erweiterungen der Firewall-Erzeugung

- **Protokolle und Ports** (offen). `access` könnte Einträge wie `{to: Server, ports: [443, 22/tcp]}` aufnehmen. Satz 10 bleibt gültig, wenn man die Bedingung der Freigaberegeln um Protokoll und Port ergänzt: Das Urteil ist dann genau dann ACCEPT, wenn ein Paar (i, j) mit passendem Dienst existiert.

- **Mengen und Objektgruppen** (umgesetzt, `-group-rules`). Die Gruppierung aus Satz 16 wird als `nftables`-Menge, `ipset` oder `object-group` (Cisco) ausgegeben.

- **Protokollierung** verworfener Pakete (umgesetzt, `-log-drops`); zeitlich begrenzte Freigaben sind offen.

- **Rückübersetzung als Test** (umgesetzt, `-roundtrip`, `roundtrip.py`). Die erzeugten Regeln werden wieder eingelesen und die daraus rekonstruierte Menge P' mit P verglichen; zusätzlich muss das Regelwerk alles Übrige verwerfen (Default-Deny erkannt). Gilt immer $P' = P$, stimmen Generator und Leser mit Satz 10 überein. Das ist ein Test und kein Beweis, denn gleiche Fehler in beiden Teilen bleiben unentdeckt.

- **Weitere Plattformen** (offen): Juniper, MikroTik, pfSense/OPNsense, Palo Alto, Cloud-Sicherheitsgruppen. Jede Plattform braucht ihre eigene Semantik-Aussage analog zu den Sätzen 10 und 11.

7.7 Verbindung zum Subgraph Algorithmus

Die Zugriffsmatrix Z ist die Adjazenzmatrix des Zugriffsgraphen G_Z (Definition 6); der Subgraph Algorithmus ist in [14] beschrieben. Im Konfigurationsbeispiel ist in der Dateireihenfolge (`Server`, `Entwicklung`, `HR`, `Drucker`, `Gäste`)

$$Z = \begin{pmatrix} 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

Die Signaturen $\sigma_j = \sum_i Z_{ij} 2^i + j \cdot 2^n$ des Subgraph Algorithmus sind für $n = 5$ gleich [2, 32, 64, 103, 128]: Spalte 0 hat nur an Zeile 1 eine Eins ($2^1 = 2$), Spalte 3 hat Einsen in den Zeilen $0, 1, 2$ ($1 + 2 + 4 = 7$, dazu $3 \cdot 32 = 96$).

Lemma 10 (Richtlinienvergleich bei gleichen Subnetzen)**.** Seien zwei Richtlinien P, P' über denselben Subnetzen $N_1, \ldots, N_n$ (alle nichtleer) gegeben. Dann nimmt die erste Regelliste jedes neue Paket an, das die zweite annimmt, genau dann, wenn $P' \subseteq P$, also $Z' \leq Z$ komponentenweise, also $G_{Z'} \subseteq G_Z$ als Subgraph.

Beweis. „⇐“: Nach Satz 10 wird ein Paket genau bei einem Paar in der Richtlinie angenommen, und $P' \subseteq P$ überträgt das. „⇒“: Gäbe es $(i,j) \in P' \setminus P$, so wähle $x \in N_i$, $y \in N_j$ (nichtleer). Die zweite Liste nimmt (x, y, NEU) an, die erste verwirft es nach Satz 10 (die Gruppen sind eindeutig), ein Widerspruch. □

Bei übereinstimmender Beschriftung ist der Vergleich ein einfacher Vergleich der Einträge in $O(n^2)$ (`matrix_included`, `compare_policies` in `policy.py`). Der Subgraph Algorithmus ist für den Fall gedacht, dass zwei Konfigurationen dieselbe Struktur in unterschiedlicher, zyklisch verschobener Gruppenreihenfolge beschreiben, etwa beim Zusammenführen von Richtlinien nach einer Umordnung. Seine Entscheidung ist dafür jedoch nicht verlässlich genug (Bemerkung 10). Auch die in der Einführung des Subgraph Algorithmus genannte Verifikation von Programmen über Syntaxbäume hat eine Parallele: Ein Plan lässt sich als Graph über Subnetz-Blöcken (Teilbaum-Beziehung nach Lemma 1) vergleichen.

Bemerkung 10 (Der Subgraph Algorithmus ist eine Heuristik)**.** Die vorliegende Fassung von `subgraph.py` berechnet für jede Spalte von Z eine Zeilensignatur und meldet „A ist in B enthalten“, wenn in einem zyklischen Fenster von B mindestens zwei aufeinanderfolgende Signaturen mit denen von A übereinstimmen (längste gemeinsame zusammenhängende Teilfolge der Länge mindestens 2). Sie vergleicht Spalten also auf *Gleichheit* und nicht auf Teilmengenbeziehung. Daraus folgen zwei Fehlerarten:

(a) Ein enthaltener Graph wird übersehen. Für $A = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \leq B = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$ gilt $Z_A \leq Z_B$, der Algorithmus entscheidet aber `keep_both`.

(b) Ein nicht enthaltener Graph wird als enthalten gemeldet. Für $A = \begin{pmatrix} 0 & 0 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix}$ und $B = \begin{pmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$ hat A die Kante $(1,2)$, die B fehlt; die Signaturfolgen $(2,0,2)$ und $(2,0,0)$ haben aber die gemeinsame Teilfolge $(2,0)$, und der Algorithmus entscheidet `equal_keep_A`, also „gleich“.

An je 3000 zufälligen Paaren (drei bis sechs Knoten, Kantendichte 0,4, festes Startelement) wurde $A \subseteq B$ in 1884 Fällen erkannt und in 1116 (37,2 %) übersehen; bei unabhängigen Paaren meldete der Algorithmus in 262 Fällen (8,7 %) fälschlich Enthaltensein (`pruefe-erweiterungen.py`). Die Zahlen gelten für die vorliegende Datei; sie beschreiben nicht den Algorithmus, wie er beabsichtigt war.

Für Firewall-Richtlinien hat das Folgen: Wer aufgrund einer falschen Meldung „enthalten“ die vermeintlich allgemeinere Richtlinie behält, ändert die Menge der erlaubten Zugriffe. Deshalb ist in `policy.py` die exakte Prüfung maßgeblich. Sie umfasst (i) den Vergleich der Paarmengen nach Gruppennamen (Lemma 10) und (ii) die exakte Entsprechung der Rotationssuche: Z_A ist *zyklisch* in Z_B enthalten ab Position s, wenn $Z_A \leq Z_B[\pi_s, \pi_s]$ für die Indizes $\pi_s = (s, s+1, \ldots, s+n_A-1) \bmod n_B$ gilt. Das kostet $O(n_B\, n_A^2)$ und ist `rotation_containment`. `compare_policies` gibt beides zusammen mit der Rohentscheidung des Subgraph Algorithmus zurück und kennzeichnet in `subgraph_agrees`, ob die

Heuristik mit der exakten Prüfung übereinstimmt; die Empfehlung „behalte die allgemeinere“ (`keep`) folgt nur aus der exakten Prüfung. Der Aufruf `subnetplan a.yaml -compare b.yaml` gibt den Bericht auf der Fehlerausgabe aus.

7.8 Integration und Ausgabeformate

- **DHCP:** Konfiguration für ISC DHCP oder Kea direkt aus Netz, Gateway, erster und letzter Host.
- **DNS:** Zonen für die Rückauflösung; bei Netzen kleiner als `/24` ist die klassenlose Delegation nach [9] nötig.
- **IPAM:** Import in NetBox oder phpIPAM; Export als Infrastruktur-Code (Terraform, Ansible).
- **Switch-Konfiguration:** VLAN-Datenbank und Schnittstellen nach IEEE 802.1Q [10].
- **Weitere Eingabeformate:** JSON-Schema für Editor-Unterstützung, Import aus Tabellenkalkulation, grafische Oberfläche und Web-Dienst.
- **Warnungen** (umgesetzt, `-warnings`, `-vpn-network`, `-warn-threshold`): Basisnetz nicht aus privaten Bereichen [3], Überschneidung mit VPN-Netzen, Auslastung über einem Schwellwert, bei Cisco fehlender Rückverkehr.

7.9 Qualitätssicherung und formale Verifikation

- **Eigenschaftsbasierte Tests** (umgesetzt, ohne zusätzliche Bibliothek): Die Sätze 4, 2, 8 und 10 sowie die der Erweiterungen sind als seeded Zufallstests in `tests/test_properties.py` dauerhaft im Projekt; dazu kommen die Skripte `pruefe-saetze.py` und `pruefe-erweiterungen.py`. Die Bibliothek Hypothesis wäre eine mögliche Ergänzung, weil sie Gegenbeispiele automatisch verkleinert.
- **Mutationstests:** Zeigen, ob die 247 Tests Änderungen des Codes tatsächlich bemerken; 100 % Abdeckung ist dafür nur eine notwendige Bedingung.
- **SMT-Verifikation:** Die Invarianten (K1) bis (K3) lassen sich für beschränktes n als Bitvektor-Probleme in einem SMT-Löser formulieren und vollständig beweisen.
- **Maschinengeprüfter Beweis:** Der Induktionsbeweis von Satz 2 ist kurz und eignet sich für einen Beweisassistenten (Lean, Isabelle, Coq).
- **Differenztests** gegen andere Werkzeuge (`ipcalc`, `sipcalc`, `netaddr`) und Fuzzing des YAML-Parsers.

7.10 Offene theoretische Fragen

- **Online-Variante:** Gruppen treffen nacheinander ein, und Entscheidungen sind endgültig. Wie viel Platz geht gegenüber der Offline-Optimallösung (Satz 5) im schlimmsten Fall verloren? Das Buddy-System [11] bietet die natürliche Strategie.

– **Minimale Umnummerierung:** Die Planung mit einer Kostenfunktion für verschobene Hosts bei Änderung der Anforderungen.

– **Mehrere Ziele:** Gleichzeitige Minimierung von Verschnitt, Routing-Einträgen (Aggregation, Satz 19) und Regelzahl (Satz 16).

– **Komplexität mit Nebenbedingungen:** Reihenfolge-Vorgaben (Gruppen müssen in bestimmter Reihenfolge liegen) bei fester Ausrichtung: Wie ändert sich die Aussage von Satz 3, wenn die Reihenfolge teilweise vorgeschrieben ist?

7.11 Priorisierte Übersicht

Vorschlag	Aufwand	Nutzen	Begründung	Stand
Bereinigung `access`, Warnungen	gering	Fehler früh erkennen	Satz 15	umgesetzt
Validierung $O(n)$, Cisco $O(n+a)$	gering	Skalierung	Satz 13	umgesetzt
Cisco-Rückverkehr	mittel	korrekte Funktion	Satz 11	umgesetzt
Exakte Arithmetik	gering	Robustheit	Lemma 5	umgesetzt
IPv6-`/64`, Mindestgrößen	gering	Betriebspraxis	Satz 18	umgesetzt
Regelgruppierung	mittel	kürzere Regeln	Satz 16	umgesetzt
Bestandsnetze, Pools	mittel	Praxistauglichkeit	Satz 17	umgesetzt
Inkrementelle Planung	mittel	keine Umnummerierung	Korollar 1	umgesetzt
Hierarchie	mittel	Routen-Aggregation	Satz 19	umgesetzt
Eigenschaftsbasierte Tests	mittel	Gewissheit	Abschnitt 4.6	umgesetzt
Ports, weitere Plattformen	hoch	Funktionsumfang	—	offen
Formale Verifikation (SMT, Beweisassistent)	hoch	Gewissheit	Satz 2	offen

7.12 Implementierungshinweis

Der Subgraph Algorithmus ist verfügbar unter `https://github.com/naphz86/subgraph` [14]. Der Code von `sbntgg` befindet sich im Projekt `sbntgg` (Paket `subnetplan`, Version 1.1.0, Python 3.11 oder neuer, Abhängigkeiten PyYAML, NumPy und das Paket `subgraph`, das `pip` über die `pyproject.toml` installiert). Installation und Aufruf:

```
python -m venv .venv && source .venv/bin/activate   # Windows: .venv\Scripts\activate
pip install -e ".[dev]"
subnetplan examples/config.example.yaml --verify --roundtrip --warnings
subnetplan examples/config.example.yaml --firewall nftables --group-rules
subnetplan examples/config.example.yaml --format json -o plan.json
subnetplan neue.yaml --extend plan.json
subnetplan a.yaml --compare b.yaml
pytest
```

Literaturverzeichnis

[1] J. Mogul, J. Postel: *Internet Standard Subnetting Procedure*, RFC 950, 1985.

[2] T. Pummill, B. Manning: *Variable Length Subnet Table For IPv4*, RFC 1878, 1995.

[3] Y. Rekhter et al.: *Address Allocation for Private Internets*, RFC 1918, 1996.

[4] V. Fuller, T. Li: *Classless Inter-domain Routing (CIDR): The Internet Address Assignment and Aggregation Plan*, RFC 4632, 2006.

[5] R. Hinden, S. Deering: *IP Version 6 Addressing Architecture*, RFC 4291, 2006.

[6] A. Retana et al.: *Using 31-Bit Prefixes on IPv4 Point-to-Point Links*, RFC 3021, 2000.

[7] M. Kohno et al.: *Using 127-Bit IPv6 Prefixes on Inter-Router Links*, RFC 6164, 2011.

[8] B. Carpenter et al.: *Analysis of the 64-bit Boundary in IPv6 Addressing*, RFC 7421, 2015.

[9] H. Eidnes, G. de Groot, P. Vixie: *Classless IN-ADDR.ARPA delegation*, RFC 2317, 1998.

[10] IEEE: *IEEE Standard for Local and Metropolitan Area Networks – Bridges and Bridged Networks (802.1Q)*.

[11] D. E. Knuth: *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*, Abschnitt 2.5 (dynamische Speicherzuteilung, Buddy-System), Addison-Wesley.

[12] M. R. Garey, D. S. Johnson: *Computers and Intractability*, W. H. Freeman, 1979 (Problem GT18).

[13] J. Orlin: *Contentment in graph theory: covering graphs with cliques*, Indagationes Mathematicae 80, 1977.

[14] S. Epp: *Der Subgraph Algorithmus*, 2026. Quelle: `https://github.com/naphz86/subgraph`.